

CareFlow AI

Implementation Prompts

Step-by-step build prompts for the CareFlow AI healthcare scheduling platform.

Coding-agent prompt pack · PROMPT1–PROMPT50 · 50 prompts

Source	prompts.txt
Prompts	50 sections (PROMPT1–PROMPT50)
Generated	2026-09-20
Format	A4 · justified body · numbered sections

About this document

IMPORTANT INSTRUCTION Use these prompts one at a time, in order. Each prompt is intended to be given to the coding agent independently. The agent must inspect the existing repository before changing anything, preserve working functionality, avoid unnecessary rewrites, and implement only the requested scope.

Contents

Prompt 1	4
Prompt 2	7
Prompt 3	10
Prompt 4	14
Prompt 5	17
Prompt 6	21
Prompt 7	24
Prompt 8	27
Prompt 9	31
Prompt 10	35
Prompt 11	40
Prompt 12	43
Prompt 13	46
Prompt 14	49
Prompt 15	52
Prompt 16	55
Prompt 17	59
Prompt 18	62
Prompt 19	65
Prompt 20	69
Prompt 21	75
Prompt 22	79
Prompt 23	83
Prompt 24	87
Prompt 25	91
Prompt 26	95
Prompt 27	97

Prompt 28	99
Prompt 29	102
Prompt 30	105
Prompt 31	109
Prompt 32	112
Prompt 33	115
Prompt 34	118
Prompt 35	121
Prompt 36	125
Prompt 37	129
Prompt 38	132
Prompt 39	136
Prompt 40	139
Prompt 41	143
Prompt 42	149
Prompt 43	153
Prompt 44	156
Prompt 45	159
Prompt 46	165
Prompt 47	168
Prompt 48	171
Prompt 49	174
Prompt 50	177

Prompt 1

You are working on the CareFlow AI project, a multi-tenant healthcare access, scheduling, patient intake, AI-agent, EHR-integration, workflow, and operations platform.

Your task is to establish and/or cleanly organize the repository foundation for the complete architecture described below.

Do NOT blindly create a new project if an existing repository already contains implementation. First inspect the complete repository structure and understand what already exists. Reuse existing working code and structure wherever possible.

TARGET ARCHITECTURE

The project should be organized around:

- Backend: FastAPI modular monolith.
- Frontend: independent React + Vite applications for:
 - Patient
 - Doctor
 - Hospital Admin
 - Operations Admin
 - Platform Admin
- Database: PostgreSQL.
- Cache/context and asynchronous infrastructure: Redis.
- Background processing: Celery worker and Celery Beat.
- AI: separate AI orchestration layer.
- Capabilities: controlled in-process capability/MCP layer.
- Domain services: business logic independent of AI.
- Scheduling: independent scheduling service.
- Integration: EHR connector abstraction with Mock EHR.
- Reliability: verification, synchronization, reconciliation.
- Observability: audit events, correlation IDs, capability execution tracking, workflow execution tracking.
- Infrastructure: Docker and docker-compose.

Create a clean top-level structure similar to:

careflow-ai/ backend/ frontend/ apps/ patient/ doctor/ hospital-admin/ operations-admin/ platform-admin/ infra/ docs/ tests/

The exact structure may differ if the existing repository already has a better implementation. Do not move or rename large numbers of existing files merely for aesthetics.

BACKEND ORGANIZATION

The backend should be prepared for logical separation such as:

backend/app/ main.py core/ api/ domain/ scheduling/ mcp_server/ ai/ integration/ reliability/ workflow/ notification/ voice/ observability/

The purpose of these boundaries is:

- core: configuration, database, security, dependencies, tenancy, audit.
- api: health and shared API concerns.
- domain: business entities and services.
- scheduling: availability and booking-slot logic.
- mcp_server: controlled capabilities used by AI and other authorized callers.
- ai: orchestration and conversation context.
- integration: EHR connector abstraction and implementations.
- reliability: verification, synchronization, reconciliation.
- workflow: Celery and event-driven processing.
- notification: notification delivery.
- voice: web/telephone voice integration.
- observability: correlation, tracing, metrics, audit.

FRONTEND ORGANIZATION

Each application should remain independently runnable.

Expected applications:

- 1 Patient
- 2 Doctor
- 3 Hospital Admin
- 4 Operations Admin
- 5 Platform Admin

Do not introduce unnecessary monorepo tooling if the existing project does not use it.

DOCUMENTATION

Create or preserve a docs directory for:

- architecture
- AI documentation
- integration documentation
- deployment documentation
- API documentation where useful

QUALITY REQUIREMENTS

Before making changes:

- 1 Inspect the repository.
- 2 Identify existing apps, backend modules, Docker files, environment files, tests, and deployment files.
- 3 Do not duplicate an existing module.
- 4 Do not delete working functionality.
- 5 Do not introduce hypothetical services that are not needed.
- 6 Keep the architecture implementation-grounded.
- 7 Prefer small, reviewable changes.

After the structural work:

- Verify imports where applicable.

- Verify each frontend application has a clear entry point.
- Verify backend application entry point is identifiable.
- Verify the repository can be understood by another developer.
- Add or update minimal documentation describing the structure.
- Run relevant existing tests or validation commands.

IMPORTANT: This task is repository foundation only. Do not implement the complete healthcare workflow, AI agent, EHR connector, scheduling engine, voice system, or dashboards yet unless required merely to preserve already-existing functionality.

Prompt 2

You are continuing work on the existing CareFlow AI repository.

Your task is to establish the backend application foundation using FastAPI while preserving all existing working backend functionality.

First, Inspect The Existing Code.

Before changing anything:

- 1 Inspect backend/app and all existing backend modules.
- 2 Find the current FastAPI application entry point.
- 3 Identify existing routers.
- 4 Identify existing configuration/environment handling.
- 5 Identify existing middleware.
- 6 Identify existing exception handling.
- 7 Identify existing health endpoints.
- 8 Identify existing tests.
- 9 Determine which parts are already implemented and reuse them.

Do not replace an existing working backend with a new skeleton.

TARGET BACKEND FOUNDATION

The backend should use:

- FastAPI
- Pydantic
- SQLAlchemy
- Alembic
- environment-based configuration
- structured application startup
- clear router registration
- health/readiness endpoints
- centralized error handling where appropriate

Create or preserve an application entry point similar in responsibility to:

backend/app/main.py

The FastAPI application should:

- create the application instance
- register middleware
- register routers
- expose health/readiness endpoints
- initialize only what is appropriate for application startup
- avoid putting business logic inside main.py

CONFIGURATION

Create or preserve a centralized configuration layer.

Configuration should support environment variables for values such as:

- DATABASE_URL
- REDIS_URL
- JWT configuration
- AI provider configuration
- EHR configuration
- CORS origins
- environment/development mode

Do not hardcode secrets.

Provide a safe .env.example containing variable names and non-secret examples only.

HEALTH ENDPOINTS

Ensure there is at least a basic endpoint:

GET /health

It should return a simple successful response when the application itself is healthy.

If readiness checks are implemented, keep them separate from liveness.

For example:

GET /health GET /ready

Do not make health endpoints unnecessarily dependent on every external provider.

MIDDLEWARE FOUNDATION

Preserve or establish a clean location for:

- CORS
- correlation ID
- request context
- error handling

Correlation IDs must eventually allow a booking or other important operation to be traced across services and workflows.

Do not implement the entire observability system here. Establish the application boundary needed for later work.

ROUTER ORGANIZATION

Routers should remain separate from domain services.

Use a pattern similar to:

router ? service ? database/integration

Do not put database-heavy business logic directly inside route functions.

TESTING

Create or preserve basic backend tests for:

- application startup
- /health

- configuration loading where practical
- a simple API request

Use the project's existing testing framework if one already exists.

VALIDATION

After implementation:

- 1 Start the backend.
- 2 Verify /health.
- 3 Verify the application imports successfully.
- 4 Run relevant tests.
- 5 Confirm existing API routes were not accidentally removed.
- 6 Report exactly what was changed.

IMPORTANT: Do not implement authentication, scheduling, AI, MCP capabilities, EHR integration, questionnaires, workflows, or dashboards in this task unless they already exist and must be preserved.

Prompt 3

You are continuing work on the existing CareFlow AI repository.

Your task is to establish and/or complete the PostgreSQL database foundation using SQLAlchemy and Alembic.

First, Inspect Before Changing.

Inspect:

- existing SQLAlchemy setup
- existing Base class
- engine/session configuration
- models
- Alembic configuration
- existing migrations
- database environment variables
- test database setup
- SQLite usage in tests, if present

Do not recreate models or migrations that already exist.

DATABASE ARCHITECTURE

The production database must be PostgreSQL.

The architecture must support explicit entities for the major platform concepts, including:

- User
- Hospital
- Hospital Admin/Staff
- Department
- Specialty
- Doctor
- Calendar
- Availability
- Blocked Slot
- Patient profile/preferences
- Appointment
- Appointment history
- Questionnaire
- Questionnaire response
- AI conversation/context where applicable
- Capability
- Capability execution
- Healthcare-system connection
- External identifier mapping

- Integration operation
- Integration verification
- Reconciliation record
- Workflow
- Workflow execution
- Notification
- AI evaluation
- Audit event
- Operational event

Do NOT implement all of these if they already exist. This task is about establishing the database foundation and ensuring the model/migration system is reliable.

SQLALCHEMY FOUNDATION

Ensure there is a clean database layer responsible for:

- engine creation
- session creation
- Base model
- transaction boundaries
- dependency injection for sessions where appropriate

Keep database infrastructure separate from domain services.

ALEMBIC

Ensure Alembic can:

- 1 connect to the configured database
- 2 detect/import models correctly
- 3 create migrations
- 4 upgrade to latest migration
- 5 downgrade where supported
- 6 maintain alembic_version correctly

Do not manually modify migration history to hide errors.

TENANT-AWARE DESIGN

The database design must support hospital-scoped data.

Hospital-owned entities should have an explicit hospital relationship where appropriate.

The future tenant isolation mechanism will use the authenticated user's hospital scope rather than trusting arbitrary client-provided hospital IDs.

Do not implement the entire RBAC/tenant authorization system here, but make sure the database structure supports it.

Appointment-Ready Database Design

The database foundation must be able to support:

- appointment state
- idempotency key

- correlation ID
- external appointment ID
- appointment history
- doctor ID
- patient ID
- hospital ID
- start/end timestamps

Scheduling and booking logic will be implemented later.

INDEXING

Establish appropriate indexes where they are already clearly justified, especially for fields that will be frequently queried such as:

- hospital_id
- doctor_id
- date/time
- correlation_id
- idempotency_key

Do not add dozens of speculative indexes.

TEST DATABASE

If the repository already uses SQLite for tests, preserve that compatibility where practical.

If PostgreSQL-specific features are required in production, make their test behavior explicit rather than silently creating inconsistent semantics.

MIGRATION SAFETY

Never delete production migrations simply to make Alembic pass.

If migrations already exist:

- inspect them
- preserve their history
- identify inconsistencies
- create corrective migrations when required

VALIDATION

After implementation:

- 1 Start PostgreSQL.
- 2 Run Alembic upgrade to head.
- 3 Verify tables are created.
- 4 Verify a database session can connect.
- 5 Run database-related tests.
- 6 Verify existing migrations remain valid.
- 7 Confirm no secrets are committed.

At the end, report:

- database structure created/changed

- migrations created/changed
- tests executed
- any remaining database issues

IMPORTANT: Do not implement the full scheduling engine, EHR integration, AI, questionnaires, or workflow engine here. Only create the database foundation needed for those later components.

Prompt 4

You are continuing work on the existing CareFlow AI repository.

Your task is to establish the Redis and Celery asynchronous-processing foundation.

First, Inspect The Existing Repository.

Check:

- Redis configuration
- Celery configuration
- existing Celery tasks
- event bus implementation
- workflow models
- Docker/compose files
- environment variables
- existing tests
- any existing AI context storage

Do not create duplicate Celery applications, duplicate Redis clients, or competing queue systems.

TARGET ARCHITECTURE

Redis has two major responsibilities in this architecture:

- 1 Short-lived AI conversation/context storage where applicable.
- 2 Celery broker/backend for asynchronous workflows.

Celery is responsible for long-running and asynchronous work such as:

- notifications
- reminders
- EHR retrievals
- reconciliation
- synchronization
- workflow execution
- scheduled sweeps

Do NOT introduce Kafka or another message broker unless the existing repository already requires it and there is a documented reason.

CELERY FOUNDATION

Create or preserve a clear Celery application such as:

backend/app/workflow/celery_app.py

It should support:

- broker configuration
- result backend configuration where required
- task autodiscovery or explicit task registration

- retry configuration
- safe task execution
- worker execution
- Celery Beat scheduling

REDIS CONFIGURATION

Use an environment variable such as:

REDIS_URL

Do not hardcode Redis credentials or hostnames.

The application should work with:

- local docker-compose Redis
- hosted Redis
- Railway/production Redis

AI CONTEXT

If AI context storage already exists, preserve it.

The context layer should be capable of storing short-lived structured conversational state such as:

- selected hospital
- selected doctor
- selected slot
- current appointment
- current booking stage
- pending action
- relevant preferences

Do not turn Redis into the source of truth for durable appointment/business state.

PostgreSQL remains the source of truth for durable business records.

EVENT BUS

Establish or preserve an event bus abstraction so domain actions can publish events without directly depending on Celery implementation details.

Conceptually:

Domain Service ? publish_event() ? WorkflowExecution record ? Celery ? Task

The event bus must not make core booking dependent on successful asynchronous processing.

If an appointment is successfully persisted, the API should not unnecessarily fail just because a reminder or notification worker is temporarily unavailable.

WORKFLOW EXECUTION

Where the project already has a WorkflowExecution model, use it.

It should eventually support statuses such as:

- running
- completed
- failed

- retrying

Do not redesign existing workflow models unless required.

CELERY BEAT

Prepare scheduled execution for future tasks such as:

- reminder sweeps
- reconciliation sweeps
- cleanup of expired temporary data where appropriate

Keep the schedule centralized.

TESTING

Add or preserve tests for:

- Celery application import
- task registration
- Redis configuration
- eager task execution in tests where appropriate
- event publication
- workflow execution creation

Do not require a real external Redis server for every unit test if the project already supports eager/test modes.

DOCKER COMPATIBILITY

Ensure the local infrastructure can run:

- Redis
- backend
- Celery worker
- Celery Beat

Do not combine worker and API into one process for local development unless the existing design explicitly does so.

VALIDATION

After implementation:

- 1 Start Redis.
- 2 Start the backend.
- 3 Start a Celery worker.
- 4 Start Celery Beat if configured.
- 5 Execute a simple test task.
- 6 Verify the task reaches the worker.
- 7 Verify relevant workflow records where applicable.
- 8 Run the automated tests.

IMPORTANT: Do not implement the complete workflow engine, appointment reminders, EHR retry logic, AI context behavior, or notification system yet. Establish the reliable asynchronous infrastructure that those later components will use.

Prompt 5

You are continuing work on the existing CareFlow AI repository.

Your task is to establish a reproducible Docker-based local development environment for the CareFlow AI architecture.

First, Inspect The Existing Infrastructure.

Check:

- Dockerfiles
- docker-compose files
- alternate compose files
- backend startup scripts
- frontend configuration
- PostgreSQL configuration
- Redis configuration
- Celery worker configuration
- Celery Beat configuration
- existing health checks
- environment templates
- Railway/deployment configuration

Do not replace working infrastructure simply to make it look different.

Target Local Development Environment

The local environment should support the core architecture:

Frontend applications ? FastAPI backend ? PostgreSQL + Redis ? Celery worker + Celery Beat

A sensible local compose structure is:

services:

- postgres
- redis
- backend
- worker
- beat

Frontend Vite applications may run directly on the host during development if that is already how the repository works.

DATABASE SERVICE

PostgreSQL should:

- use a persistent Docker volume
- expose the expected local port where appropriate
- initialize with environment variables

- have a health check
- use a non-secret development password from environment configuration

Do not commit real credentials.

REDIS SERVICE

Redis should:

- have a persistent or appropriate development configuration
- expose the expected local port where needed
- have a health check
- be addressable by backend and Celery using a Docker service name

BACKEND SERVICE

The backend container should:

- install dependencies
- expose the FastAPI port
- receive environment variables
- connect to PostgreSQL
- connect to Redis
- expose /health
- run migrations when appropriate, but do not make startup scripts dangerously destructive

If the existing project already has a startup script, inspect and improve it rather than replacing it.

CELERY WORKER

Create/preserve a dedicated worker service that:

- uses the same backend code
- connects to Redis
- connects to PostgreSQL
- starts the Celery worker
- logs task execution clearly

CELERY BEAT

Create/preserve a dedicated Beat service that:

- connects to the same Celery configuration
- loads scheduled tasks
- avoids multiple Beat instances during local development
- logs scheduled execution

NETWORKING

All backend infrastructure services should communicate through the Docker Compose network.

The backend should connect to:

PostgreSQL: postgres://...

Redis: redis://redis:6379/...

Do not use localhost inside containers to refer to another container.

ENVIRONMENT MANAGEMENT

Create or preserve:

`.env.example`

Document:

- DATABASE_URL
- REDIS_URL
- JWT settings
- CORS settings
- AI provider keys
- EHR configuration
- SMTP configuration
- voice/telephony configuration
- any other required runtime configuration

Never put actual secrets into:

- Dockerfile
- `docker-compose.yml`
- source code
- documentation
- Git

DEVELOPMENT EXPERIENCE

The goal is that a developer can clone the repository, create a local `.env` file, and start the core infrastructure with a simple command such as:

`docker compose up --build`

If frontend apps run separately, document the commands clearly.

HEALTH CHECKS

Use health checks for:

- PostgreSQL
- Redis
- backend where practical

Make service dependencies wait for required infrastructure to become healthy instead of relying purely on arbitrary sleep commands.

MIGRATIONS

Ensure the development startup process can apply Alembic migrations safely.

Do not automatically destroy or recreate the database on every startup.

Existing developer data must not be wiped by normal startup.

TESTING

After implementation:

- 1 Build all relevant Docker images.
- 2 Start the compose stack.
- 3 Confirm PostgreSQL is healthy.
- 4 Confirm Redis is healthy.
- 5 Confirm backend is healthy.
- 6 Confirm Celery worker is running.
- 7 Confirm Celery Beat is running.
- 8 Run a simple background task.
- 9 Run backend tests.
- 10 Verify an existing frontend can communicate with the backend.

TROUBLESHOOTING OUTPUT

At the end, provide:

- files created/changed
- services available
- ports
- environment variables required
- commands to start
- commands to stop
- commands to inspect logs
- tests executed
- any known limitations

IMPORTANT: Do not implement new product features in this task. The goal is a reliable, reproducible development infrastructure for the CareFlow AI architecture.

FINAL QUALITY RULE

All five prompts must be executed against the real existing repository, not against an imagined greenfield project.

For every change:

- inspect first
- reuse existing implementation
- avoid duplication
- preserve working behavior
- keep architecture boundaries clear
- add tests
- validate locally
- report exact changes

Prompt 6

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete the identity and user foundation required by the CareFlow AI architecture.

First, Inspect Before Changing.

Inspect:

- existing User model
- authentication models and schemas
- password hashing implementation
- JWT utilities
- auth routers
- current roles
- hospital relationship on User
- patient profile implementation
- doctor/user relationship
- existing migrations
- frontend authentication flows
- existing tests

Do not recreate an authentication system if one already exists. Extend and correct the existing implementation.

TARGET USER MODEL

The platform has four primary product roles:

- platform_admin
- hospital_admin
- doctor
- patient

The implementation may also have an operations-admin role if the repository already uses it for operational responsibilities. Do not silently remove an existing operational role.

The User entity should support:

- id
- email
- password_hash
- role
- hospital_id where applicable
- is_active
- created_at
- updated_at

Use explicit relationships/foreign keys where appropriate.

PATIENT PROFILE

Keep patient-specific information separate from the generic authentication identity.

A patient profile may contain:

- name
- contact information
- date of birth
- communication preferences
- relevant profile information
- external patient ID where applicable

Follow data minimization. Do not add sensitive healthcare fields just because they might be useful later.

DOCTOR IDENTITY

Doctor authentication should remain separate from doctor domain configuration.

The User record represents identity/authentication.

The Doctor record represents provider/business information.

Do not duplicate password or authentication fields inside Doctor.

HOSPITAL ADMIN

Hospital administrators must be associated with a specific hospital where applicable.

Do not allow a hospital admin to select an arbitrary hospital ID during ordinary requests and thereby change their tenant.

PASSWORD SECURITY

Use the existing secure password hashing implementation if present.

If it is missing, implement Argon2-based password hashing.

Never store plaintext passwords.

Do not log passwords or authentication secrets.

JWT

Implement or preserve:

- access tokens
- refresh tokens
- token type
- subject/user ID
- expiration

The application must distinguish access tokens from refresh tokens.

Protected endpoints must reject refresh tokens when an access token is required.

AUTH ENDPOINTS

Ensure the required authentication operations exist or remain functional:

- POST /auth/register
- POST /auth/login
- POST /auth/refresh

- GET /auth/me

If the repository already has additional auth endpoints, preserve them.

REGISTRATION RULES

Patient self-registration should be straightforward.

Hospital admin and platform admin creation must not be opened as unrestricted public registration unless the existing product explicitly requires it.

Doctor accounts should normally be created/invited through hospital administration workflows.

Do not implement the full hospital onboarding workflow yet unless it already exists.

TESTING

Test:

- password hashing
- password verification
- registration
- login
- invalid credentials
- inactive users
- access token creation
- refresh token creation
- wrong token type
- /auth/me
- duplicate email
- role persistence

SECURITY

Never trust role or hospital membership supplied by a client request when creating an authenticated context.

The database user record will later be authoritative for tenant scope and role.

VALIDATION

After implementation:

- 1 Run migrations.
- 2 Create a test user.
- 3 Register/login.
- 4 Call /auth/me.
- 5 Test invalid credentials.
- 6 Test inactive user behavior.
- 7 Run relevant unit/integration tests.
- 8 Verify no secrets or plaintext passwords are stored/logged.

Do not implement scheduling, AI, EHR, voice, questionnaires, or workflows in this task.

Prompt 7

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete role-based authorization for the application.

FIRST, INSPECT:

- current auth dependency functions
- current role checks
- router-level authorization
- user context creation
- frontend route guards
- existing operations/platform roles
- tests
- hospital ownership checks

Do not introduce a second authorization mechanism if one already exists.

TARGET AUTHORIZATION MODEL

The authenticated request should resolve a trusted application context containing at least:

- user_id
- role
- hospital_id where applicable

The authorization layer should expose reusable mechanisms similar in responsibility to:

get_current_context() require_role(...)

Keep authentication and authorization separate:

Authentication: "Who is this user?"

Authorization: "Is this user allowed to perform this operation?"

ROLE RESPONSIBILITIES

Platform Admin:

- manage/approve hospitals
- access platform-level operational information
- monitor platform-wide activity
- manage platform configuration where supported

Hospital Admin:

- manage their hospital
- manage doctors
- configure services
- configure appointment types
- configure calendars and availability
- configure questionnaires

- manage integrations/workflows for their hospital

Doctor:

- access their own provider data
- manage their own availability/calendar
- view authorized appointments
- view authorized questionnaire responses

Patient:

- manage their own profile
- access their own appointments
- access their own questionnaires
- manage their own preferences
- perform allowed booking/rescheduling/cancellation operations

OPERATIONS ADMIN

If the repository already has an operations-admin role, define its permissions explicitly for:

- reconciliation
- workflow operations
- integration failures
- operational monitoring
- retry/resolve operations

Do not automatically give operations-admin unrestricted platform-admin access.

ROUTER AUTHORIZATION

Protected routers should use centralized authorization.

Avoid repeating raw role strings and authorization logic throughout route handlers.

Prefer:

`router ? authentication context ? require_role(...) ? service`

RESOURCE AUTHORIZATION

Role authorization alone is insufficient.

For example:

A hospital admin may have permission to update doctors, but only doctors belonging to their own hospital.

A doctor may access appointments, but only appointments they are authorized to see.

A patient may access appointments, but only their own appointments.

Implement ownership/resource checks through services or shared authorization helpers where appropriate.

DO NOT TRUST CLIENT INPUT

Do not authorize a request merely because the request body contains:

`hospital_id = current user's hospital`

The server must derive or validate tenant ownership from trusted authenticated context and database relationships.

ERRORS

Use appropriate HTTP semantics:

- unauthenticated ? 401
- authenticated but unauthorized ? 403
- missing resource ? 404 where appropriate

Do not expose sensitive information through authorization errors.

FRONTEND

Inspect all existing frontend applications.

Ensure frontend route guards improve UX but are NOT treated as the security boundary.

The backend must enforce authorization independently.

TESTING

Create tests for:

- platform admin access
- hospital admin access
- doctor access
- patient access
- cross-role denial
- inactive user denial
- cross-hospital denial
- patient accessing another patient's resource
- doctor accessing another hospital's data
- hospital admin accessing another hospital's data
- operations-admin restrictions if applicable

VALIDATION

Run:

- backend tests
- authorization tests
- relevant frontend tests if available

Confirm existing valid API operations still work.

Do not implement the full tenant filtering mechanism yet if it is not already part of this repository task. Establish the reusable authorization boundary that the tenant layer will use next.

Prompt 8

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete hospital-scoped multi-tenancy and enforce tenant data isolation.

This is a security-critical task.

FIRST, INSPECT:

- User.hospital_id
- Hospital model
- current authentication context
- existing tenant helpers
- all hospital-scoped domain models
- existing router queries
- service-layer queries
- platform-admin behavior
- database constraints/indexes
- tests for cross-hospital access

Do not assume that existing filtering is correct. Trace actual database queries.

CORE SECURITY RULE

Hospital A must never access Hospital B's private data.

This applies to:

- doctors
- calendars
- availability
- appointments
- questionnaires
- questionnaire responses
- AI activity
- conversations/context where tenant-owned
- integrations
- workflow records where tenant-owned
- notifications
- operational records
- patient information belonging to another tenant

PLATFORM ADMIN

Platform admin is the explicit cross-tenant role.

Platform admin may access platform-wide records where the product requires it.

Do not accidentally give ordinary hospital admins platform-wide access.

TRUSTED TENANT CONTEXT

The authenticated User row in PostgreSQL is authoritative.

Do not trust a hospital_id claim supplied by the client.

If the JWT contains hospital_id, treat it only as non-authoritative metadata if the existing architecture retains it.

For protected requests:

- 1 Decode and validate the access token.
- 2 Load the User from PostgreSQL.
- 3 Confirm the user is active.
- 4 Resolve role and hospital_id from the database.
- 5 Build trusted RequestContext.
- 6 Apply tenant filtering.

TENANT HELPER

Create or preserve a reusable mechanism with responsibility similar to:

hospital_scoped_query(query, context)

Behavior:

- platform_admin ? platform-wide query where authorized
- hospital-scoped user ? WHERE hospital_id = context.hospital_id
- patient/doctor ? additional ownership restrictions

Do not silently fall back to an unscoped query.

DATA MODEL

Identify which entities are:

- 1 Global/platform-owned.
- 2 Hospital-owned.
- 3 User-owned.
- 4 Doctor-owned.
- 5 Appointment-owned.
- 6 Operational/global.

Do not blindly add hospital_id to every table.

For entities whose tenant is derivable through another entity, use the appropriate relationship while ensuring the query cannot cross tenant boundaries.

SERVICE LAYER

Tenant isolation should be enforced in the service/data access path, not only in frontend filters.

Example:

Hospital Admin ? Router ? Service ? Tenant-scoped query ? PostgreSQL

Do not rely on the frontend to hide another hospital's records.

Cross-Tenant Write Protection

A hospital admin must not be able to:

- assign another hospital's doctor

- create an appointment for another hospital through manipulated IDs
- attach a questionnaire from another hospital
- modify another hospital's calendar
- use another hospital's integration configuration

Validate related resources belong to the same authorized tenant.

PATIENT OWNERSHIP

Patients should only access their own private data.

A patient must not be able to retrieve another patient's:

- appointments
- profile
- questionnaires
- questionnaire responses
- AI context
- notifications

DOCTOR OWNERSHIP

Doctors should only access records they are authorized to access.

At minimum, validate:

- doctor belongs to the user's hospital
- doctor identity matches where an operation is self-scoped
- appointment belongs to the authorized doctor/hospital

TESTING

This task requires strong negative tests.

Create tests proving:

- 1 Hospital A admin can read Hospital A data.
- 2 Hospital A admin cannot read Hospital B data.
- 3 Hospital A admin cannot modify Hospital B data.
- 4 Doctor A cannot access Doctor B's private records unless explicitly authorized.
- 5 Patient A cannot access Patient B's data.
- 6 Platform admin can access authorized cross-tenant records.
- 7 A forged hospital_id in a request cannot bypass tenant isolation.
- 8 A forged JWT hospital_id cannot bypass database-derived scope.
- 9 Inactive users cannot access tenant data.

Run the tests against the actual database layer used by the application.

AUDIT

Where a sensitive cross-tenant access attempt is denied, preserve the existing audit/security behavior if available.

Do not log raw healthcare data unnecessarily.

VALIDATION

After implementation:

- inspect representative queries
- run migrations if required
- run all tenant/security tests
- manually test one cross-tenant access scenario
- confirm platform admin behavior
- confirm frontend behavior still works

Do not implement hospital onboarding or scheduling in this task.

Prompt 9

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete correlation IDs, audit events, and the foundation for operational traceability.

FIRST, INSPECT:

- existing middleware
- correlation ID utilities
- audit models
- capability execution models
- appointment history
- workflow execution models
- integration operation models
- tracing utilities
- existing observability endpoints
- tests

Do not create duplicate audit or tracing systems.

Core Traceability Requirement

A major booking should eventually be traceable across:

Conversation ? AI decision ? Capability ? Scheduling ? Appointment ? EHR operation ? Verification ? Synchronization ? Workflow ? Notification

Use a correlation ID to connect related operations.

Use an operation ID for individual capability/integration/workflow executions where appropriate.

CORRELATION ID MIDDLEWARE

Create or preserve middleware responsible for:

- 1 Reading X-Correlation-ID from the incoming request where appropriate.
- 2 Generating a safe ID when absent.
- 3 Attaching it to the request context.
- 4 Returning it in the response.
- 5 Making it available to downstream services.

Do not trust arbitrary user-supplied IDs for authorization.

The correlation ID is for tracing, not security.

REQUEST CONTEXT

The request/application context should eventually support:

- user_id
- role
- hospital_id
- correlation_id

Do not create a giant global state object.

AUDIT EVENT

Create or preserve an audit event model capable of recording important actions such as:

- login/access
- appointment operations
- patient-data access
- AI actions
- capability execution
- integration operations
- configuration changes
- administrative actions
- security-relevant failures

Include appropriate metadata such as:

- event ID
- timestamp
- actor/user ID where appropriate
- hospital ID where appropriate
- action
- resource type
- resource ID
- correlation ID
- operation ID
- success/failure
- safe metadata

Do not store unnecessary raw healthcare content.

SENSITIVE DATA

Audit records must be privacy-aware.

Do NOT automatically dump:

- full patient conversations
- full questionnaire answers
- passwords
- access tokens
- API keys
- medical details
- raw voice/audio

into logs or audit records.

Record enough metadata to explain what happened without unnecessarily storing sensitive content.

CAPABILITY EXECUTION

If the repository already has capability execution tracking, preserve it.

Each capability execution should eventually support:

- tool/capability name
- actor
- correlation ID
- operation ID
- latency
- success/failure
- safe error classification
- timestamp

Denied capability calls should remain auditable if the existing design requires this.

TRACE ENDPOINT

If an observability endpoint already exists, ensure it can retrieve events related to a correlation ID without exposing unrelated tenant data.

For example:

GET /observability/trace/{correlation_id}

It should present an ordered operational trace such as:

- 1 request
- 2 AI action
- 3 capability call
- 4 appointment operation
- 5 EHR operation
- 6 verification
- 7 workflow
- 8 notification

Do not expose sensitive payloads unnecessarily.

TRACING UTILITY

Create or preserve a safe tracing/span helper for internal operations.

It should:

- record start/end
- record duration
- record safe attributes
- capture errors
- never cause the business operation to fail merely because tracing failed

OBSERVABILITY PRINCIPLE

Observability must be secondary to business correctness.

If an audit write fails, the application should follow the repository's explicit policy rather than silently pretending the audit succeeded.

Do not swallow failures blindly.

TESTING

Test:

- generated correlation ID
- propagated correlation ID
- response header
- supplied correlation ID behavior
- audit creation
- capability execution recording
- trace retrieval
- tenant isolation in trace retrieval
- sensitive information is not logged
- failures are recorded safely

VALIDATION

Run existing tests and add focused observability tests.

Then manually execute a representative API request and verify:

request ? correlation ID ? audit event ? response correlation ID

Do not implement the complete analytics dashboard in this task.

Prompt 10

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete the Hospital domain and the hospital onboarding lifecycle.

This is the first major tenant-management business workflow.

FIRST, INSPECT:

- existing Hospital model
- hospital schemas
- hospital routers
- hospital service
- User/Hospital relationship
- hospital migrations
- platform-admin functionality
- hospital-admin frontend
- audit implementation
- tenant isolation implementation
- existing tests

Do not recreate hospital CRUD if it already exists. Reuse it and fill only the missing requirements.

HOSPITAL ENTITY

A hospital should support appropriate fields such as:

- id
- organization/hospital name
- address
- contact information
- status
- created/updated timestamps
- configuration needed by later modules

Only add fields that are required by the existing implementation or the requirements.

HOSPITAL LIFECYCLE

Implement or preserve this lifecycle:

Draft ? Submitted ? Under Review ? Approved ? Active operation

Alternative paths:

Under Review ? Rejected

Rejected ? Resubmitted ? Under Review

Approved ? Suspended ? Reinstated

Use explicit state transitions rather than arbitrary status updates.

ROLE RESPONSIBILITIES

Hospital registration:

- creates a draft/submitted hospital application as appropriate
- captures required organization information
- associates the initial hospital administrator where appropriate

Platform Admin:

- view applications
- start review
- approve
- reject
- request corrections
- suspend
- reinstate
- view hospital activity

Hospital Admin:

- manage only their own hospital
- configure hospital information
- cannot approve their own hospital
- cannot access another hospital

APPROVAL RULE

Only an approved hospital should be able to progress into production-capable operations such as:

- active doctors
- published availability
- receiving appointments
- production healthcare-system integration

Do not allow an unapproved hospital to bypass the lifecycle by directly modifying status through a generic CRUD endpoint.

State Transition Validation

Create explicit service methods with responsibilities similar to:

submit_hospital() start_review() approve_hospital() reject_hospital() request_corrections() suspend_hospital()
reinstate_hospital()

Each transition should:

- validate the current state
- validate the actor's role
- validate tenant/platform scope
- update the state
- create appropriate audit history
- preserve correlation ID where available

INVALID TRANSITIONS

Reject invalid transitions.

Examples:

- draft ? approved without review
- patient ? approve hospital
- hospital admin ? approve their own hospital
- rejected ? active without resubmission/review
- suspended ? active without reinstatement process

Use clear errors.

Hospital Configuration Preparation

The hospital domain should be ready for later configuration of:

- departments
- specialties
- services
- appointment types
- consultation duration
- calendars
- availability
- questionnaires
- communication preferences
- workflows
- healthcare-system configuration

Do not fully implement all of those configuration modules yet unless they already exist.

DOCTOR CREATION GATE

Ensure the service layer can enforce:

Only approved/active hospitals may create or activate doctors.

Do not rely only on frontend UI restrictions.

APPOINTMENT GATE

Prepare the domain rules so later appointment creation can verify:

- hospital is approved/active
- doctor belongs to hospital
- doctor is active
- scheduling is configured

Do not implement the complete appointment system in this task.

AUDITABILITY

Important lifecycle actions must be auditable:

- submitted
- review started

- approved
- rejected
- correction requested
- suspended
- reinstated

Use the correlation/audit infrastructure from the previous work.

FRONTEND

Implement or preserve the minimum hospital onboarding UI required to demonstrate:

Hospital Admin:

- create/register hospital
- view application status
- view hospital profile

Platform Admin:

- list applications
- view application
- approve
- reject
- request corrections
- suspend/reinstate where appropriate

Keep the UI focused on the workflow. Do not build the complete hospital administration dashboard yet.

API

Use clear domain endpoints consistent with the existing API conventions.

Do not expose a generic "set any status" endpoint.

TESTING

Add tests for:

- hospital creation
- submission
- review
- approval
- rejection
- correction request
- resubmission
- suspension
- reinstatement
- invalid transitions
- role restrictions
- cross-tenant access
- unapproved hospital cannot activate doctors
- unapproved hospital cannot receive appointments

- audit event creation

Run migrations and all relevant tests.

END-TO-END VALIDATION

Demonstrate:

Hospital Admin ? Register hospital ? Submit ? Platform Admin ? Review ? Approve ? Hospital becomes eligible for configuration

At the end, report:

- existing hospital functionality reused
- files changed
- API changes
- database/migration changes
- frontend changes
- tests executed
- any remaining limitations

IMPORTANT:

Do not move ahead into doctor configuration, scheduling, AI, EHR, questionnaires, or workflow implementation except for minimal gates/interfaces needed by the hospital lifecycle.

Prompt 11

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete the hospital configuration foundation after hospital onboarding has been established.

First, Inspect Before Changing.

Inspect:

- Hospital model and service
- hospital configuration modules
- existing Department model
- existing Specialty model
- existing Service model
- existing AppointmentType model
- hospital admin routes
- existing migrations
- tenant-scoping helpers
- existing hospital-admin frontend
- tests

Do not recreate existing entities. Reuse and extend the current implementation.

GOAL

An approved hospital must be able to configure the information that later drives provider discovery, scheduling, questionnaires, workflows, and patient booking.

The hospital configuration should remain hospital-scoped.

CORE CONFIGURATION AREAS

Prepare the hospital domain to support:

- hospital profile
- departments
- specialties
- services
- appointment types
- consultation duration
- communication preferences
- workflow configuration
- healthcare-system configuration

Do not implement the complete scheduling engine, questionnaires, workflow engine, or EHR connector in this task.

APPROVAL GATE

Only hospitals in an allowed operational state should be able to publish production-facing configuration.

Do not allow a rejected/suspended hospital to behave like an active hospital.

TENANCY

Every hospital administrator request must be restricted to the administrator's own hospital.

A hospital admin must not be able to:

- create configuration for another hospital
- modify another hospital's department
- read another hospital's specialties
- attach another hospital's service
- manipulate another hospital's appointment types

Validate tenant ownership in the backend.

CONFIGURATION SERVICES

Keep configuration business logic in services rather than route functions.

Use a structure similar to:

router ? hospital configuration service ? tenant-scoped database query

VALIDATION

Configuration should reject invalid data such as:

- empty required names
- invalid durations
- invalid references
- duplicate records where uniqueness is required
- resources belonging to another hospital
- configuration changes by unauthorized roles

AUDIT

Important configuration changes should be auditable.

Record safe metadata such as:

- actor
- hospital
- resource type
- resource ID
- action
- correlation ID
- timestamp

Do not store unnecessary sensitive healthcare content in audit logs.

TESTING

Test:

- approved hospital configuration
- unauthorized configuration
- cross-tenant configuration attempts

- invalid configuration
- duplicate configuration
- inactive/suspended hospital restrictions
- audit creation

Run existing tests and migrations.

IMPORTANT:

Do not implement doctor availability or appointment booking yet. The goal is to make hospital configuration a reliable foundation for the next modules.

Prompt 12

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete hospital departments, specialties, and services.

FIRST, INSPECT:

- existing Hospital model
- Department model
- Specialty model
- Service model
- hospital configuration services
- API routes/schemas
- frontend configuration screens
- migrations
- tenant authorization
- existing tests

Reuse existing implementation whenever possible.

DOMAIN MODEL

A hospital may contain:

Hospital ??? Departments ??? Specialties ??? Services

All hospital-specific records must belong to a hospital.

DEPARTMENT

A Department should support appropriate fields such as:

- id
- hospital_id
- name
- description where required
- active/enabled state where useful
- timestamps

Do not add unnecessary clinical fields.

SPECIALTY

A Specialty should support:

- id
- hospital_id
- name
- description where required
- active/enabled state where useful
- timestamps

A specialty must not accidentally become globally shared if the existing product treats it as hospital configuration.

SERVICE

If the existing requirements/model support hospital services, implement:

- id
- hospital_id
- name
- description where appropriate
- active state
- timestamps

Do not confuse a service with an appointment type.

RELATIONSHIPS

Where appropriate, support relationships such as:

Doctor ? Department ? Specialty

and hospital ownership:

Hospital ? Department ? Specialty ? Service

Use the existing domain model rather than introducing duplicate relationships.

CRUD

Implement or preserve:

- create
- list
- retrieve
- update
- deactivate/delete according to existing project conventions

Prefer soft deactivation where deleting a configured resource could break historical records.

TENANT ISOLATION

Hospital Admin:

- can manage only their hospital

Platform Admin:

- can manage platform-wide records where authorized

Doctor/Patient:

- should not receive hospital configuration write access

Cross-tenant resource IDs must be rejected.

DISCOVERY READINESS

These entities will later be used by patient discovery and AI capabilities.

Ensure list/search responses expose only information appropriate for discovery.

Do not expose private hospital configuration.

VALIDATION

Prevent:

- duplicate names within the same hospital where uniqueness is required
- empty names
- invalid hospital references
- modification by unauthorized users
- use of resources from another hospital

TESTING

Test:

- 1 Create department.
- 2 List department.
- 3 Update department.
- 4 Deactivate department.
- 5 Create specialty.
- 6 Create service.
- 7 Cross-hospital access fails.
- 8 Unauthorized roles fail.
- 9 Duplicate configuration is rejected.
- 10 Approved/suspended hospital behavior follows lifecycle rules.

FRONTEND

Implement or preserve a focused hospital-admin configuration screen for:

- Departments
- Specialties
- Services

Keep it functional rather than visually elaborate.

Do not implement provider scheduling or AI discovery yet.

Prompt 13

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete appointment-type configuration for hospitals.

FIRST, INSPECT:

- existing AppointmentType model
- hospital configuration models
- appointment schemas
- scheduling code if already present
- hospital-admin routes
- patient-facing booking code
- migrations
- tests

Do not create a duplicate appointment type system.

PURPOSE

Appointment types define what kinds of appointments a hospital can offer.

Examples may include administrative categories such as:

- consultation
- follow-up
- new patient visit
- virtual consultation
- in-person consultation

Use the existing product terminology where it already exists.

APPOINTMENT TYPE DATA

An appointment type may require:

- id
- hospital_id
- name
- description
- duration_minutes
- consultation_mode where applicable
- active/enabled state
- timestamps

Only add fields that are required by the current repository/requirements.

DURATION

Duration is important because later scheduling will use appointment type duration to calculate bookable slots.

Validate:

- duration exists when required
- duration is positive
- duration is within a reasonable configured range
- duration does not contain invalid values

Do not hardcode one global appointment duration.

CONSULTATION MODE

If the existing product supports consultation modes, keep them explicit rather than embedding them inside free-form text.

Examples:

- in-person
- virtual

Do not invent additional modes unless the existing repository requires them.

HOSPITAL OWNERSHIP

Every appointment type must belong to a hospital.

Hospital admins may manage appointment types only for their own hospital.

A platform admin may manage them only where explicitly authorized.

Patients and doctors should not receive unrestricted configuration write access.

Doctor Compatibility Preparation

Later, scheduling must determine whether a doctor supports a given appointment type.

Prepare the model/service boundary for compatibility checks.

Do not implement the complete doctor compatibility system yet unless it already exists.

ACTIVE/INACTIVE

Do not hard-delete an appointment type that may already be referenced by historical appointments.

Prefer deactivation where appropriate.

An inactive appointment type must not be offered for new bookings.

Historical appointments must remain understandable.

API

Provide or preserve clear operations for:

- create
- list
- retrieve
- update
- activate/deactivate

Avoid a generic endpoint that accepts arbitrary hospital_id and trusts it.

The server must derive tenant scope from authenticated context.

FRONTEND

Hospital Admin should be able to:

- see appointment types
- create one
- set duration
- configure supported mode where applicable
- activate/deactivate
- edit configuration

Patient-facing UI should only display active types that are valid for the relevant hospital/doctor.

Do not build the complete patient booking flow yet.

TESTING

Test:

- create appointment type
- validation of duration
- duplicate names if uniqueness applies
- activation/deactivation
- cross-tenant access
- unauthorized writes
- inactive type excluded from future-booking queries
- historical references remain valid

Run migrations and all relevant tests.

IMPORTANT:

Do not implement slot calculation here. The next scheduling work will consume appointment type duration and compatibility.

Prompt 14

You are continuing work on the existing CareFlow AI repository.

Your task is to establish the hospital-level communication, workflow, and healthcare-system configuration boundaries needed by later modules.

FIRST, INSPECT:

- hospital configuration models
- notification configuration
- workflow configuration
- integration/EHR configuration
- environment/configuration classes
- existing hospital-admin APIs
- existing secrets handling
- migrations
- tests

Do not store secrets directly in ordinary hospital configuration tables unless the existing security architecture explicitly supports secure secret storage.

COMMUNICATION PREFERENCES

A hospital should be able to configure appropriate communication preferences such as:

- supported notification channels
- enabled/disabled notification types
- reminder behavior where already supported
- contact information

Keep actual notification execution in the notification/workflow layers.

Do not put email/SMS sending code into hospital configuration services.

WORKFLOW CONFIGURATION

Prepare a hospital-scoped configuration boundary for future workflows.

A configuration may identify:

- enabled workflow
- trigger/event
- enabled state
- timing/delay configuration
- applicable hospital scope

Do not build a full workflow engine here.

The workflow engine will later execute configured workflows asynchronously.

Healthcare-System Configuration

A hospital must be able to indicate that an external healthcare system is configured.

Keep vendor-specific behavior behind the EHR connector abstraction.

The hospital configuration layer must not contain:

- vendor-specific appointment algorithms
- direct EHR HTTP calls
- AI logic
- scheduling logic

Use a structure similar to:

Hospital Configuration ? Integration Configuration ? EHRConnector ? External Healthcare System

SECRET HANDLING

Never hardcode:

- API keys
- database passwords
- EHR credentials
- AI provider keys
- Twilio credentials
- SMTP credentials

Do not expose secrets in:

- GET configuration endpoints
- frontend responses
- logs
- audit events

If secure secret storage is not yet implemented, store only non-secret configuration and create a clear interface for secure credentials to be added later.

INTEGRATION STATUS

Where appropriate, configuration should expose safe status information such as:

- configured
- not configured
- enabled
- disabled
- connection status

Do not expose raw credentials.

HOSPITAL SCOPE

Every configuration read/write must respect hospital tenant boundaries.

Platform/operations roles should only access configuration according to explicit authorization.

AUDIT

Record important configuration changes:

- communication settings changed
- workflow configuration changed

- integration configuration changed
- integration enabled/disabled

Avoid logging secret values.

TESTING

Test:

- hospital admin can configure their hospital
- another hospital cannot access it
- unauthorized roles are rejected
- secrets are never returned
- secret-like values are not written to logs
- configuration changes create audit events
- suspended hospital restrictions are enforced where applicable

FRONTEND

Provide a simple Hospital Admin configuration area for:

- communication preferences
- workflow settings where currently supported
- healthcare-system configuration/status

Do not build the complete workflow editor or EHR integration UI yet.

IMPORTANT:

This task establishes configuration boundaries only. Actual EHR calls, workflow execution, notification sending, and scheduling remain separate services.

Prompt 15

You are continuing work on the existing CareFlow AI repository.

Your task is to make the Hospital Admin configuration experience coherent and production-ready enough to support the next provider/scheduling work.

First, Inspect The Existing Frontend And Backend.

Review:

- hospital-admin React application
- routing
- authentication context
- API client
- existing hospital pages
- hospital configuration endpoints
- departments
- specialties
- services
- appointment types
- communication configuration
- integration configuration
- existing loading/error patterns
- existing UI component system
- backend authorization
- tests

Do not replace the existing frontend architecture or introduce a new UI framework.

GOAL

Create one coherent Hospital Admin configuration experience covering the functionality already implemented.

Suggested navigation:

Hospital Admin ??? Overview ??? Hospital Profile ??? Departments ??? Specialties ??? Services ??? Appointment Types ??? Communication ??? Integrations

Use the application's existing design language.

HOSPITAL PROFILE

Allow the hospital admin to view/edit only permitted hospital profile information.

Display application status clearly.

Do not allow the UI to expose unauthorized lifecycle controls.

DEPARTMENTS

Provide:

- list

- create
- edit
- deactivate
- loading state
- validation errors
- API errors

SPECIALTIES

Provide equivalent management.

SERVICES

Provide equivalent management.

APPOINTMENT TYPES

Provide:

- name
- duration
- consultation mode where supported
- active/inactive state
- edit/create behavior

Do not allow inactive appointment types to appear as available booking options.

COMMUNICATION

Display and manage supported communication configuration without exposing secrets.

INTEGRATION

Display safe integration configuration/status.

Never render:

- API secrets
- database passwords
- access tokens
- AI keys
- EHR credentials

as ordinary UI values.

SECURITY

Frontend route guards should ensure only hospital-admin users reach this application area, but backend authorization remains the actual security boundary.

Do not trust hospital_id stored in frontend state.

Use the authenticated API context.

UX

Every configuration page should have:

- loading state

- empty state
- success feedback
- validation errors
- API failure handling
- disabled state while saving
- confirmation for destructive/deactivation actions where appropriate

Do not overbuild the UI.

The objective is a reliable administration experience, not a design showcase.

API CLIENT

Reuse the existing axios/API client.

Do not create multiple API clients for the same backend.

Use typed request/response models if the existing TypeScript architecture supports them.

TESTING

Test:

- protected routes
- configuration listing
- creation
- update
- deactivation
- error states
- cross-tenant backend rejection
- unauthorized backend access
- secret values not displayed

Run:

- backend tests
- frontend tests if available
- type checking
- build

END-TO-END CHECK

Verify this complete configuration journey:

Approved Hospital ? Hospital Admin Login ? Hospital Profile ? Department ? Specialty ? Service ? Appointment Type ? Communication/Integration Configuration

The configuration must persist after page refresh.

Do not implement doctors, calendars, availability, booking, AI, voice, questionnaires, or workflows beyond the configuration boundaries already established.

FINAL RULE

Preserve the existing CareFlow AI architecture. Do not introduce a new architectural pattern merely to complete the UI. The backend domain/service boundaries, tenant isolation, authorization, audit, and configuration ownership must remain the source of truth.

Prompt 16

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete the Doctor domain and doctor lifecycle.

First, Inspect Before Changing.

Inspect:

- existing User model and doctor relationship
- Doctor model
- doctor schemas
- doctor router/service
- hospital configuration
- hospital approval state
- department/specialty/service models
- appointment type configuration
- tenant authorization
- audit infrastructure
- hospital-admin frontend
- doctor frontend
- migrations
- existing tests

Do not recreate an existing Doctor implementation. Reuse and improve what is already present.

DOCTOR DOMAIN

The Doctor entity should support appropriate information such as:

- id
- user_id
- hospital_id
- name
- photo where supported
- specialty
- department
- qualifications
- experience
- languages
- consultation types
- appointment duration where applicable
- external provider ID
- status
- timestamps

Use the existing schema terminology when it differs.

Do not store authentication credentials inside Doctor if authentication already belongs to User.

HOSPITAL OWNERSHIP

Every doctor must belong to a hospital.

A hospital admin can create/manage doctors only inside their own hospital.

A platform admin may manage doctors only where explicitly authorized.

A doctor must not be able to move themselves to another hospital.

PATIENT DISCOVERY

The Doctor domain should eventually support discovery by:

- hospital
- department
- specialty
- service
- active status
- supported consultation mode

Do not implement the complete AI discovery system here.

DOCTOR LIFECYCLE

Implement or preserve:

Invited ? Active ? Inactive

and where supported:

Active ? Suspended

Only active doctors should be eligible for future appointment scheduling.

STATUS TRANSITIONS

Use explicit service methods rather than a generic status update endpoint.

Examples:

invite_doctor() activate_doctor() deactivate_doctor() suspend_doctor() reinstate_doctor()

Validate:

- actor role
- hospital ownership
- hospital lifecycle state
- current doctor state
- required doctor information

APPROVED HOSPITAL GATE

Only an approved/operational hospital should be able to activate doctors.

A draft, rejected, or suspended hospital must not be able to publish an active provider.

EXTERNAL PROVIDER ID

Support an external provider identifier for EHR mapping.

Do not treat the external provider ID as the internal doctor primary key.

Do not call the EHR directly from the Doctor service.

Appointment Type Compatibility

Prepare the Doctor domain to represent which appointment types or consultation modes a doctor can support.

If the repository already has this relationship, reuse it.

If it does not, establish the cleanest model/service boundary without implementing the complete scheduling engine yet.

AUDIT

Record important doctor lifecycle actions:

- invited
- activated
- deactivated
- suspended
- reinstated
- profile/configuration changes where appropriate

Do not log unnecessary sensitive information.

API

Implement or preserve clear operations for:

- create/invite
- retrieve
- list
- update
- activate
- deactivate
- suspend
- reinstate

Do not expose unrestricted hospital_id/status mutation.

TESTING

Test:

- doctor creation
- hospital ownership
- doctor activation
- invalid lifecycle transitions
- unapproved hospital restrictions
- suspended hospital restrictions
- cross-tenant access
- doctor self-access restrictions
- external provider ID persistence
- appointment-type compatibility where implemented

- audit events

FRONTEND

Ensure the hospital admin can manage doctor records through the existing UI architecture.

Keep the UI focused on doctor administration.

Do not implement calendar/availability logic in this task.

At the end:

- run migrations if needed
- run backend tests
- run frontend checks
- report exact changes
- report any remaining limitations

Prompt 17

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete the Doctor Administration experience.

FIRST, INSPECT:

- existing doctor backend APIs
- doctor schemas
- hospital-admin application
- doctor application
- authentication/role guards
- hospital tenant context
- existing UI component system
- API client
- doctor lifecycle
- existing tests

Do not create a second doctor-management frontend flow.

Hospital Admin Doctor Management

The Hospital Admin should be able to:

- view doctors belonging to their hospital
- search/filter doctors
- create/invite a doctor
- edit doctor information
- activate a doctor
- deactivate a doctor
- suspend/reinstate where supported
- view doctor status
- view specialty/department
- view external provider ID only if safe and required

Do not expose credentials or secrets.

DOCTOR FORM

Support appropriate fields already defined by the backend, including:

- name
- photo where supported
- specialty
- department
- qualifications
- experience
- languages

- consultation types
- supported appointment types where implemented
- external provider ID where appropriate

Use backend validation as the source of truth.

Do not duplicate complicated business rules in the frontend.

STATUS UX

Make lifecycle state visible.

Examples:

Invited Active Inactive Suspended

Only show actions valid for the current state.

Do not show an "activate" action when the hospital itself is not eligible to operate.

LOADING/ERROR STATES

Every major operation should have:

- loading state
- disabled submit while saving
- validation feedback
- API failure feedback
- success feedback
- empty state
- retry behavior where appropriate

TENANT SECURITY

The frontend must not assume that changing a `hospital_id` makes a doctor belong to another hospital.

The backend remains authoritative.

Do not put hospital IDs into editable form fields unless they are display-only and required.

DOCTOR APPLICATION

If the Doctor SPA already exists, ensure the authenticated doctor can access only their authorized profile and future schedule-related areas.

Do not implement the complete calendar UI yet.

ROUTING

Use the existing React Router structure.

Do not add a new routing library.

API CLIENT

Reuse the existing API client.

Use existing authentication interceptors/token refresh behavior.

Do not create a separate HTTP client for doctor management.

TESTING

Test:

- hospital admin route access
- doctor list
- doctor create/invite
- doctor edit
- activation
- deactivation
- unauthorized access
- cross-tenant backend denial
- invalid forms
- loading/error states
- doctor self-access restrictions

Run:

- type checking
- frontend build
- backend tests
- relevant frontend tests

END-TO-END CHECK

Verify:

Approved Hospital ? Hospital Admin ? Doctors ? Create Doctor ? Doctor Invited ? Activate ? Doctor becomes eligible for scheduling

Do not implement calendar/availability/booking in this task.

Prompt 18

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete the doctor calendar foundation.

FIRST, INSPECT:

- existing Calendar model
- scheduling service
- Doctor model
- hospital timezone/configuration
- existing availability implementation
- blocked slot implementation
- migrations
- doctor frontend
- hospital-admin frontend
- tests

Do not recreate a calendar system if one already exists.

GOAL

Each active doctor who participates in scheduling should have a calendar configuration that defines the scheduling boundary.

The calendar must be independent from the conversational AI.

The architecture should be:

Doctor ? Calendar ? Availability Rules ? Blocked Slots / Leave ? Scheduling Engine ? Bookable Slots

CALENDAR DATA

Use only fields needed by the current architecture, potentially including:

- id
- doctor_id
- timezone
- active/enabled state
- created_at
- updated_at

If hospital-level timezone is already authoritative, do not duplicate it unnecessarily.

TIMEZONE

Scheduling must use an explicit timezone.

Do not assume the server timezone.

The calendar should provide a reliable basis for converting user-selected local dates/times into consistent timestamps.

Do not silently mix naive and timezone-aware datetime values.

CALENDAR LIFECYCLE

Support appropriate operations such as:

- create
- activate
- deactivate
- retrieve
- update

Only authorized hospital administrators and the doctor should be able to manage appropriate calendar information.

A doctor must not modify another doctor's calendar.

A hospital admin must not modify a calendar belonging to another hospital.

Active Doctor/Hospital Gates

A calendar may exist before a doctor is active, but only an eligible active doctor with a valid active calendar should become bookable.

A suspended/inactive doctor must not appear as bookable even if a calendar exists.

A suspended hospital must not publish new availability.

WORKING HOURS PREPARATION

The calendar should provide a clean boundary for later working-hour rules.

Do not hardcode:

09:00–17:00

unless that is already explicitly configured in the repository.

Working hours will be represented by availability rules.

BLOCKED PERIODS

Do not store blocked periods directly as arbitrary calendar text.

Use a separate blocked-slot concept where appropriate.

The calendar service should later be able to combine:

calendar + availability rules - blocked slots - leave - existing appointments

TESTING

Test:

- calendar creation
- calendar retrieval
- timezone validation
- activation/deactivation
- doctor ownership
- hospital ownership
- inactive doctor restrictions
- cross-tenant access
- invalid datetime/timezone input

DATABASE

Add migrations only when necessary.

Use appropriate foreign keys and uniqueness constraints so a doctor cannot accidentally have multiple active primary calendars if the architecture expects one calendar per doctor.

FRONTEND

Prepare the Doctor UI for a Calendar/Availability section.

At this stage, provide the calendar configuration foundation only.

Do not implement the full visual time-grid or slot booking interface yet.

VALIDATION

Verify that:

Doctor ? Calendar ? Scheduling layer

is a clean dependency direction.

The calendar must not import or depend on the AI orchestrator.

Prompt 19

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete doctor availability rules and blocked periods.

FIRST, INSPECT:

- Calendar model
- existing availability models
- scheduling service
- doctor lifecycle
- appointment types
- blocked slots
- leave support if present
- timezone handling
- hospital admin UI
- doctor UI
- database migrations
- existing tests

Do not duplicate existing availability logic.

GOAL

Represent when a doctor is normally available and when that availability is overridden by blocked periods.

Core model:

Doctor ? Calendar ? Availability Rules + Blocked Slots + Leave + Appointments ? Scheduling Engine

AVAILABILITY RULES

Support recurring working-hour rules where appropriate.

A rule may represent:

- doctor
- day of week
- start time
- end time
- timezone through the calendar
- active/enabled state

Do not hardcode working hours globally.

Example:

Monday: 09:00 ? 17:00

Tuesday: 09:00 ? 17:00

The implementation must support different hours on different days.

VALIDATION

Reject:

- end time before start time
- zero-length windows
- invalid day values
- overlapping rules where the architecture does not allow them
- rules for another hospital/doctor
- invalid timezone assumptions

If split shifts are supported, model them explicitly.

Do not silently merge overlapping intervals.

BLOCKED SLOTS

Implement or preserve a blocked-slot model that can represent:

- doctor
- start datetime
- end datetime
- reason/category where appropriate
- status if needed
- created_by
- timestamps

Blocked slots must prevent generated availability during the blocked interval.

Examples:

- leave
- meeting
- personal block
- holiday
- administrative block

Do not expose unnecessary private reasons to patients.

APPOINTMENT INTERACTION

Availability must later be reduced by existing appointments.

Do not treat a working-hour rule itself as a guaranteed available slot.

The future scheduling engine must calculate:

Working Hours - Blocked Slots - Leave - Existing Appointments = Bookable Availability

APPOINTMENT TYPE DURATION

Availability generation must eventually consider appointment type duration.

For example:

30-minute appointment: 09:00–09:30 09:30–10:00

60-minute appointment: 09:00–10:00 10:00–11:00

Do not implement final slot generation here unless the repository already contains it.

DOCTOR CONTROL

Doctors should be able to manage their own availability and blocked time according to their permissions.

Hospital admins should be able to manage schedules for doctors in their hospital.

Patients must not modify provider availability.

TENANT SECURITY

Every availability/blocked-slot operation must validate:

- authenticated role
- doctor ownership
- hospital ownership
- hospital operational status where required

Never trust doctor_id/hospital_id from the client without server-side validation.

TIMEZONE

Store actual appointment/blocked timestamps consistently.

Interpret recurring working-hour times in the calendar's configured timezone.

Avoid ambiguous DST behavior.

TESTING

Test:

- create availability rule
- update availability rule
- delete/deactivate rule
- create blocked slot
- overlapping blocked slots
- blocked slot inside working hours
- blocked slot outside working hours
- cross-tenant access
- unauthorized doctor access
- invalid time ranges
- timezone behavior
- inactive/suspended doctor restrictions

FRONTEND

Provide a usable Doctor Availability interface with:

- weekly working hours
- add/edit availability rule
- blocked time
- date/time selection
- validation errors
- save state
- conflict feedback

Hospital Admin should be able to manage a doctor's availability where authorized.

Do not build the final patient booking calendar yet.

IMPORTANT:

The scheduling engine must remain independent from the AI and voice layers.

Prompt 20

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete the central Scheduling Engine and real availability calculation.

This is a critical business component.

First, Inspect The Entire Existing Scheduling Implementation.

Inspect:

- Calendar
- AvailabilityRule
- BlockedSlot
- Doctor
- Hospital
- Appointment
- AppointmentType
- scheduling service
- booking service
- existing slot endpoints
- database constraints
- transactions
- row locking
- timezone handling
- tests
- frontend availability consumers

Do not replace a working scheduling engine with a new implementation. If scheduling already exists, trace it end-to-end and fix/complete it according to the requirements.

CORE PRINCIPLE

Scheduling is the source of truth for bookable slots.

The AI must never invent availability.

The scheduling engine must calculate availability from actual application state.

A slot is bookable only when all required conditions are true.

REQUIRED CONDITIONS

A slot must satisfy:

- 1 Doctor is active.
- 2 Hospital is operational/approved.
- 3 Calendar is active.
- 4 Slot is inside a valid working-hours rule.
- 5 Slot is not blocked.

- 6 Slot is not on leave.
- 7 Slot does not overlap an existing appointment.
- 8 Appointment type is compatible with the doctor/hospital.
- 9 Consultation mode is compatible where applicable.
- 10 External scheduling constraints are satisfied where applicable.
- 11 The requested slot is within the supported scheduling range.

Do not allow frontend state to override these rules.

SLOT GENERATION

Implement a deterministic function/service with responsibility similar to:

`calculate_available_slots( doctor_id, date/range, appointment_type, consultation_mode )`

The implementation should:

- load the correct doctor/calendar
- resolve timezone
- load working-hour rules
- generate candidate intervals
- remove blocked intervals
- remove leave
- remove existing appointments
- validate appointment type duration
- return actual bookable slots

Do not perform AI calls inside the scheduling engine.

Do not perform EHR calls directly inside slot calculation unless an explicitly required external constraint interface already exists.

TIMEZONE

Use the doctor's/calendar timezone for recurring working hours.

Return unambiguous timestamps to the API.

Ensure date filtering does not shift slots incorrectly because of UTC/local-time conversion.

RANGE LIMIT

Respect the repository's configured maximum scheduling window.

If the architecture already defines a maximum such as 62 days, preserve it.

Do not allow an unrestricted availability query that can generate huge datasets.

PERFORMANCE

Avoid N+1 database queries when calculating slots for a range.

Load required data efficiently:

- doctor/calendar
- availability rules
- blocked slots
- appointments

- appointment type

Use appropriate indexes.

Do not add caching until correctness is established.

CONCURRENT BOOKING

This is critical.

Two patients may request the same slot simultaneously.

The system must ensure only one succeeds.

Use the existing transaction strategy or implement a safe strategy based on:

- database transaction
- appropriate row locking
- overlap detection
- unique constraints where applicable
- idempotency keys

Do not rely on:

"check availability" then "insert appointment"

as two independent operations without a concurrency strategy.

RESERVATION

If the existing architecture uses temporary blocked-slot reservations/holds, preserve that approach.

A reservation should:

- have an owner/reference
- have start/end
- be tied to the appointment/request where appropriate
- prevent competing bookings
- be released safely on failure/cancellation

Do not create permanent blocks accidentally.

BOOKING REVALIDATION

Availability shown to a patient is only a snapshot.

Immediately before booking:

- 1 re-check doctor/hospital eligibility
- 2 re-check the slot
- 3 acquire required lock/reservation
- 4 verify no conflicting appointment
- 5 create the appointment transactionally
- 6 generate/persist idempotency information

If the slot is no longer available, return a clear conflict result such as HTTP 409.

APPOINTMENT TYPE

Slot length must use the configured appointment type duration.

Do not allow the client to submit an arbitrary duration and bypass hospital configuration.

If the doctor does not support the appointment type, reject the request.

SLOT END BOUNDARY

A slot must fit entirely within the availability window.

Example:

Working hours: 09:00–17:00

30-minute appointment: 16:30–17:00 ? valid

45-minute appointment: 16:30–17:15 ? invalid

Do not return partially fitting slots.

BLOCKED PERIOD OVERLAP

A candidate slot is unavailable if it overlaps a blocked period.

Do not only check whether the slot start lies inside the blocked period.

The entire interval must be evaluated.

APPOINTMENT OVERLAP

Similarly, detect interval overlap correctly.

For a candidate:

[start, end)

and existing appointment:

[existing_start, existing_end)

the intervals conflict when they overlap.

Do not allow adjacent appointments to conflict unnecessarily.

API

Expose or preserve clear scheduling endpoints such as:

GET /hospitals/{hospital_id}/doctors/{doctor_id}/slots

or the repository's existing equivalent.

The endpoint must:

- enforce tenant/authorization rules
- validate date range
- validate appointment type
- return only real available slots
- never expose private patient data
- return deterministic slot information

FRONTEND

Connect the existing patient/hospital/doctor scheduling consumers to the real scheduling API.

Do not create fake frontend slots.

The UI should show:

- date
- doctor

- appointment type
- available time slots
- unavailable state
- loading state
- conflict/retry behavior

Do not make the frontend the source of truth.

TESTING

This task requires extensive tests.

Unit tests:

- working-hour calculation
- timezone conversion
- appointment duration
- blocked slot subtraction
- appointment overlap
- leave
- inactive doctor
- inactive calendar
- unsupported appointment type
- hospital state
- date-range limits

Concurrency/integration tests:

- two simultaneous booking attempts for the same slot
- only one succeeds
- second receives a conflict
- idempotent retry does not create a duplicate
- failed booking releases reservation/hold
- cancellation releases availability
- rescheduling does not leave an incorrect hold

API tests:

- valid slot query
- invalid range
- unauthorized hospital
- cross-tenant doctor
- unavailable doctor
- unavailable slot
- valid booking revalidation

END-TO-END VALIDATION

Demonstrate:

Hospital approved ? Doctor active ? Calendar active ? Working hours configured ? Blocked period configured ? Appointment type configured ? Patient requests availability ? Scheduling Engine calculates slots ? Patient selects

slot ? Booking revalidates slot ? Only one concurrent request succeeds

Important Architectural Boundary

The dependency direction must remain:

AI / Voice ? Capabilities ? Appointment / Scheduling Services ? Database / Integration

Scheduling must NOT depend on:

- AI prompts
- LLM providers
- voice providers
- frontend state

The scheduling engine is deterministic business logic and must remain independently testable.

At the end, report:

- scheduling files changed
- database/migration changes
- API changes
- frontend changes
- concurrency strategy
- tests executed
- any known limitations

Prompt 21

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete the core Appointment domain and appointment lifecycle on top of the existing hospital, doctor, calendar, availability, and scheduling foundation.

First, Inspect Before Changing.

Inspect:

- Appointment model
- Appointment schemas
- appointment router/service
- scheduling engine
- Doctor
- Patient
- Hospital
- AppointmentType
- Calendar/AvailabilityRule/BlockedSlot
- existing appointment history
- idempotency implementation
- tenant authorization
- audit/correlation infrastructure
- EHR integration if already present
- frontend appointment pages
- migrations
- tests

Do not replace working appointment logic. Trace the existing implementation and complete only what is missing.

CORE APPOINTMENT ENTITY

The appointment must connect:

- patient
- doctor
- hospital
- appointment type
- start/end time
- consultation mode where applicable
- internal state
- external appointment ID where applicable
- idempotency key
- correlation ID
- timestamps

Do not store duplicate patient/doctor/hospital information unnecessarily.

STATE MACHINE

Implement or preserve explicit appointment states such as:

Requested ? Pending ? Confirmed

Pending may transition to:

- Cancelled
- Failed
- Synchronization Pending
- Reconciliation Required

Confirmed may transition to:

- Rescheduled
- Completed
- No-show
- Cancelled where the product rules allow it

Use explicit transition validation.

Do not allow arbitrary status mutation through a generic PATCH endpoint.

APPOINTMENT HISTORY

Every meaningful state transition should create an appointment history record containing appropriate safe metadata such as:

- appointment ID
- previous state
- new state
- actor
- timestamp
- correlation ID

Do not store unnecessary sensitive conversation content.

BOOKING

Booking must always use the scheduling engine.

The appointment service must not independently invent availability.

Flow:

Patient request ? Scheduling validation ? Slot revalidation ? Reservation/transaction ? Appointment creation ? State transition ? Asynchronous follow-up where appropriate

CONCURRENCY

Preserve the scheduling engine's concurrency protection.

The appointment service must not introduce a second conflicting booking mechanism.

A booking must fail safely if the selected slot becomes unavailable between availability display and final booking.

IDEMPOTENCY

Appointment creation must support idempotency.

A repeated request with the same idempotency key must not create duplicate appointments.

Implement behavior for:

- first request
- repeated identical request
- repeated request after success
- repeated request after failure
- conflicting reuse of an idempotency key

Use database constraints as a final backstop where appropriate.

PATIENT OWNERSHIP

A patient must only be able to create/manage appointments allowed for themselves.

Do not accept an arbitrary patient_id from the client and trust it.

For patient self-booking, derive the authenticated patient identity from the request context.

Doctor/Hospital Authorization

Hospital admins may manage appointments within their authorized hospital.

Doctors may access appointments they are authorized to see.

Patients may access their own appointments.

Cross-hospital access must fail.

API

Implement or preserve clear endpoints for:

- create appointment
- list appointments
- get appointment
- cancel
- reschedule
- confirm where authorized
- complete
- no-show where authorized

Keep business logic in the appointment service.

Do not place state-machine logic directly inside route functions.

RESCHEDULING

Rescheduling must:

- 1 Identify the existing appointment.
- 2 Verify the actor is authorized.
- 3 Find a valid new slot.
- 4 Revalidate availability.
- 5 Reserve the new slot safely.
- 6 Update the appointment.
- 7 Release the old slot only when safe.
- 8 Preserve appointment history.

9 Prepare external synchronization where applicable.

Do not release the old slot before the new slot is safely secured if that could create inconsistent state.

CANCELLATION

Cancellation must:

- validate state
- validate authorization
- update state
- preserve history
- release the associated availability/hold
- trigger asynchronous side effects where appropriate

Do not immediately delete appointment records.

Historical appointment records must remain available.

TESTING

Test:

- appointment creation
- valid lifecycle transitions
- invalid transitions
- patient ownership
- doctor authorization
- hospital admin authorization
- cross-tenant denial
- idempotent creation
- duplicate booking
- cancellation
- rescheduling
- completion
- no-show
- appointment history
- correlation ID propagation
- concurrent booking attempts

END-TO-END VALIDATION

Demonstrate:

Patient ? Real available slot ? Booking ? Appointment Pending ? Appointment lifecycle ? History recorded

Do not implement the complete EHR verification flow in this task if it is not already available. Leave a clean integration boundary for the next reliability work.

Prompt 22

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete the EHR connector abstraction and Mock EHR required by the CareFlow AI architecture.

FIRST, INSPECT:

- existing EHRConnector/interface
- integration service
- Mock EHR
- appointment service
- patient/doctor mappings
- external identifier mappings
- integration operation model
- existing reliability code
- environment configuration
- migrations
- tests

Do not create a second integration architecture.

CORE ARCHITECTURE

The dependency direction must be:

AI ? Capability ? Appointment/Scheduling Service ? Integration Service ? EHR Connector ? External Healthcare System

The Appointment/Domain layer must not contain vendor-specific HTTP/API code.

EHR CONNECTOR

Define or preserve an interface with responsibilities such as:

- lookup_patient
- lookup_provider
- lookup_facility
- lookup_calendar/availability where supported
- create_appointment
- update_appointment
- cancel_appointment
- reschedule_appointment
- get_appointment
- verify_appointment

Use typed input/output contracts.

Do not leak vendor-specific response formats into domain services.

INTEGRATION SERVICE

Create or preserve an IntegrationService responsible for:

- selecting the configured connector
- mapping internal entities to external identifiers
- calling connector methods
- recording integration operations
- classifying errors
- returning normalized results

Do not put retry/reconciliation policy directly inside the connector unless the existing architecture explicitly requires it.

MOCK EHR

Implement a deterministic Mock EHR for the prototype.

It should support at minimum:

- patients
- providers
- facilities
- appointments

It should allow:

- patient lookup
- provider lookup
- appointment creation
- appointment retrieval
- appointment update/reschedule
- appointment cancellation
- appointment verification

The Mock EHR must behave like an external system rather than directly sharing the application's appointment table.

Do not implement Mock EHR as a simple alias to the internal Appointment model.

External Identifier Mapping

Support mappings:

Internal Patient ? External Patient Internal Doctor ? External Provider Internal Appointment ? External Appointment
Internal Facility ? External Facility

Mappings must be explicit.

Do not use email/name matching as the permanent identity mechanism when a mapping exists.

INTEGRATION CONFIGURATION

Use existing hospital integration configuration.

Do not hardcode credentials.

Support environment-based configuration for the mock and future connectors.

Do not expose integration secrets through API responses.

ERROR NORMALIZATION

Normalize connector failures into categories such as:

- validation error
- authentication error
- authorization error
- rate limit
- timeout
- network failure
- external outage
- mapping failure
- not found
- conflict
- unknown outcome

Do not hide the original safe error classification needed for recovery.

INTEGRATION OPERATION

Record important external operations with:

- operation ID
- correlation ID
- hospital
- appointment where applicable
- operation type
- status
- timestamps
- safe error classification
- external identifier where available

Do not log credentials or sensitive healthcare payloads.

TESTING

Mock EHR tests must cover:

- patient lookup
- provider lookup
- appointment create
- appointment retrieve
- reschedule
- cancel
- verify
- missing patient
- missing provider
- slot conflict
- validation error
- timeout simulation

- network failure simulation
- duplicate/unknown outcome setup

Connector tests must prove the domain layer does not depend on Mock EHR implementation details.

END-TO-END CHECK

Verify:

Appointment Service ? IntegrationService ? EHRConnector ? MockEHR

A different connector implementation should be replaceable without changing appointment business logic.

Prompt 23

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete external appointment verification and internal/external state synchronization.

FIRST, INSPECT:

- Appointment service/state machine
- EHR Connector
- IntegrationService
- Mock EHR
- IntegrationOperation
- ExternalIdentifierMapping
- existing verification logic
- existing synchronization logic
- appointment history
- reconciliation records
- Celery/workflow infrastructure
- tests

Do not create a second verification system.

CORE RULE

An external EHR response that claims success is NOT sufficient to tell the patient that the appointment is confirmed.

The required flow is:

Create Appointment ? External Response ? Verify External Record ? Synchronize Internal State ? Confirm Patient

VERIFICATION SERVICE

Create or preserve a service responsible for:

`verify_external_appointment()`

It should:

- 1 Identify the external appointment.
- 2 Query the external system.
- 3 Compare relevant fields.
- 4 Determine whether the external state matches the expected internal operation.
- 5 Return a normalized verification result.

Verification results should distinguish at least:

- matched
- mismatched
- not found
- unknown/error

Do not treat "HTTP 200" alone as matched.

MATCHING

For appointment verification, compare the fields necessary to establish correctness, such as:

- external appointment ID
- patient mapping
- provider mapping
- facility/hospital mapping
- start time
- end time
- appointment type where supported
- status

Do not compare unnecessary sensitive data.

STATE SYNCHRONIZATION

Implement or preserve synchronization logic that maps verified external truth into internal appointment state.

Examples:

Matched: Pending/Sync Pending ? Confirmed

Not found after an uncertain create: ? Safe retry/recovery path

Divergent record: ? Synchronization Pending / Reconciliation Required

Do not blindly overwrite internal state when external data differs.

TRUST-AND-VERIFY

The system must support:

Internal booking intent ? External operation ? External verification ? Internal synchronization

The external system is authoritative for the external record.

The application's Appointment remains the internal transactional source of truth for application state, but its external status must be verified before claiming successful external synchronization.

RESCHEDULE VERIFICATION

For rescheduling:

- 1 Identify current external appointment.
- 2 Perform external update.
- 3 Verify new external state.
- 4 Synchronize internal appointment.
- 5 Release old availability only when the operation is safely established.

CANCELLATION VERIFICATION

For cancellation:

- 1 Execute external cancellation.
- 2 Verify cancellation.
- 3 Synchronize internal state.
- 4 Release availability.
- 5 Trigger follow-up workflows.

Integration Operation Status

Track external operations such as:

- started
- succeeded
- verification_pending
- verified
- failed
- unknown
- reconciliation_required

Use the existing database model where possible.

CORRELATION

Every verification operation must preserve:

- correlation ID
- operation ID
- appointment ID
- hospital ID where appropriate

This enables later operational tracing.

FAILURE CLASSIFICATION

Distinguish:

- deterministic validation failure
- authentication/authorization failure
- retryable network failure
- timeout
- rate limit
- external outage
- unknown result
- mismatched result

Do not retry non-retryable validation failures indefinitely.

TESTING

Test:

- external success + verification match
- external success + mismatch
- external success + not found
- timeout
- network failure
- reschedule verification
- cancellation verification
- repeated verification
- correlation/operation tracking

- correct appointment state transitions

END-TO-END CHECK

Demonstrate:

Appointment Pending ? Mock EHR create ? Verification ? Matched ? Appointment Confirmed

Also demonstrate:

EHR response ? Verification mismatch ? No false confirmation

Do not implement the complete reconciliation dashboard yet.

Prompt 24

You are continuing work on the existing CareFlow AI repository.

Your task is to implement or complete failure recovery, reconciliation, and safe retry behavior for external healthcare-system operations.

This is a critical reliability component.

FIRST, INSPECT:

- existing IntegrationService
- connector errors
- verification service
- synchronization service
- ReconciliationRecord
- IntegrationOperation
- Appointment state machine
- Celery tasks
- workflow event bus
- idempotency implementation
- appointment holds/reservations
- operational APIs
- tests

Do not create another retry/recovery framework.

CORE FAILURE PRINCIPLE

Never blindly retry an external appointment operation that may have succeeded.

The system must determine external state before deciding whether another write is safe.

UNKNOWN OUTCOME FLOW

Use the following conceptual flow:

External Write ? Timeout / Network Failure ? Unknown Outcome ? Query External System ? Found + Matching ? Synchronize

or:

Unknown Outcome ? Query External System ? Not Found ? Safe same-key retry where allowed

or:

Unknown Outcome ? Query External System ? Found + Divergent ? Reconciliation Required

RECONCILIATION RECORD

Create or preserve a reconciliation record containing safe operational information such as:

- record ID
- appointment ID
- hospital ID

- operation ID
- correlation ID
- failure classification
- current resolution status
- retry count
- timestamps
- safe error details
- external identifier where appropriate

Possible statuses:

- open
- retrying
- resolved
- escalated

Do not store unnecessary sensitive healthcare payloads.

RETRY POLICY

Classify operations.

Retry may be appropriate for:

- transient network failure
- timeout
- rate limiting
- temporary external outage

Retry should not be automatic for:

- invalid patient
- invalid provider
- validation error
- authorization failure
- deterministic mapping failure

For unknown appointment creation outcomes:

- 1 Query external system.
- 2 If matching record exists, synchronize.
- 3 If no record exists, retry safely using the same idempotency/reference mechanism.
- 4 If still unresolved, create reconciliation record.

Do not generate a new logical appointment identity for every retry.

CELERY

Use the existing Celery infrastructure for long-running recovery.

Do not block the API request waiting for repeated external retries.

The initial API should record the state appropriately and let asynchronous processing continue where the product flow supports it.

WORKFLOW STATES

Keep internal appointment state and operational recovery state distinct.

For example:

Appointment: Pending / Synchronization Pending / Reconciliation Required

Operational record: open / retrying / resolved / escalated

Do not combine all state into one generic status field.

SLOT/HOLD RECOVERY

If an appointment temporarily holds a slot:

- preserve the hold while external state is unknown when necessary
- release it only when the final external outcome is known
- do not release the wrong patient's hold
- do not leave permanent blocks after recovery

Make hold release idempotent.

OPERATOR ACTIONS

Prepare backend operations for authorized operators:

- retry
- resolve
- escalate

An operator must not be able to arbitrarily mark an appointment confirmed without the appropriate verification/reconciliation logic.

AUDIT

Record:

- retry
- reconciliation created
- reconciliation resolved
- escalation
- operator action

Preserve correlation and operation IDs.

TESTING

Required tests:

- 1 EHR timeout.
- 2 Network failure.
- 3 External record found after timeout.
- 4 External record not found.
- 5 Safe same-key retry.
- 6 Retry succeeds.
- 7 Retry fails again.
- 8 External record divergent.

- 9 Reconciliation created.
- 10 Operator retry.
- 11 Operator resolve.
- 12 Escalation.
- 13 Slot hold released correctly.
- 14 Duplicate retry does not create duplicate appointment.
- 15 Reconciliation action is tenant-authorized.

FAILURE DEMONSTRATION

Create a deterministic Mock EHR failure mode that can reproduce:

Booking ? EHR Timeout ? Unknown Outcome ? External Query ? Appointment Found ? Synchronize ? Confirmed

Also support a mode where:

Booking ? Timeout ? Not Found ? Safe Retry ? Success

Do not hide the failure in tests. The purpose is to prove recovery behavior.

Prompt 25

You are continuing work on the existing CareFlow AI repository.

Your task is to integrate the appointment, scheduling, EHR, verification, and recovery layers into one reliable booking workflow.

FIRST, INSPECT:

- Patient APIs
- Doctor APIs
- Hospital APIs
- Appointment service
- Scheduling engine
- EHR connector
- IntegrationService
- verification service
- synchronization service
- reconciliation system
- Celery/event bus
- audit/correlation infrastructure
- existing patient frontend
- existing doctor frontend
- tests

Do not rewrite these modules. This task is an integration task.

TARGET BOOKING FLOW

The complete backend flow should become:

Patient Request ? Hospital/Doctor Selection ? Real Availability ? Slot Selection ? Appointment Request ? Immediate Slot Revalidation ? Reservation/Transaction ? Internal Appointment Creation ? EHR Integration ? External Verification ? Internal Synchronization ? Confirmed Appointment ? Asynchronous Follow-up

If EHR integration is unavailable or optional, follow the repository's configured behavior and make the state explicit. Never falsely claim external confirmation.

Booking Service Orchestration

Create or preserve a clean service boundary that coordinates the process.

The orchestration layer may call:

Scheduling Service Appointment Service Integration Service Verification Service Synchronization Service Workflow/Event Bus

It must NOT contain:

- SQL queries scattered throughout orchestration
- vendor-specific EHR logic
- LLM calls

- voice logic
- frontend logic

TRANSACTION BOUNDARY

Internal appointment persistence and slot reservation must have a clear transactional strategy.

Do not keep a database transaction open while performing a slow external HTTP request unless the existing architecture explicitly requires it and can safely tolerate it.

Prefer a state machine that records the internal operation and lets external processing continue asynchronously when appropriate.

BOOKING STATES

Use explicit states such as:

Requested ? Pending ? Synchronization Pending ? Confirmed

Failure branches:

Pending ? Failed

Synchronization Pending ? Reconciliation Required

Do not expose internal operational complexity unnecessarily to patients, but do not hide the actual state from operators.

PATIENT EXPERIENCE

The patient should receive a response appropriate to the actual state.

Examples:

Confirmed: "Your appointment is confirmed."

Pending/external verification: "The booking is being verified."

Failure: "The appointment could not be confirmed."

Do not tell the patient an appointment is confirmed before required external verification.

CONCURRENT BOOKINGS

The integrated workflow must preserve the scheduling concurrency protection.

Test:

Patient A ? 3:00 PM

Patient B ? 3:00 PM

Only one appointment succeeds.

The other receives a conflict and must be able to choose another available slot.

IDEMPOTENCY

A network retry from the frontend or client must not create a second appointment.

Use the same idempotency key for a logical booking request.

Test repeated requests with:

- same key/same payload
- same key/different payload
- different key/same slot

- retry after timeout
- retry after external success

AUDIT TRACE

A successful booking must produce a traceable chain using correlation/operation IDs:

Request ? Appointment ? Scheduling ? EHR Operation ? Verification ? Synchronization ? Workflow Event

The exact implementation should reuse the existing audit/observability infrastructure.

ASYNC FOLLOW-UP

After successful confirmation, publish an event such as:

appointment.booked

The event may trigger future work such as:

- questionnaire assignment
- notification
- reminder scheduling
- analytics
- audit

Do not implement every downstream feature here. Establish the event integration point.

DOCTOR VISIBILITY

Once an appointment reaches the appropriate state, the Doctor application should be able to retrieve authorized appointments.

The doctor must not see another hospital's appointments.

PATIENT VISIBILITY

The Patient application should be able to:

- view upcoming appointment
- view appointment details
- view status
- view history

Only the patient's own appointments should be returned.

TESTING

Create an integration test covering the complete normal path:

Hospital ? Active Doctor ? Calendar ? Availability ? Patient ? Slot ? Appointment ? Mock EHR ? Verification ? Synchronization ? Confirmed ? Doctor can see appointment ? Patient can see appointment

Create another integration test for the failure path:

Appointment ? EHR Timeout ? Unknown Outcome ? External Verification ? Recovery ? Correct Final State

Also test:

- double booking
- idempotency
- cross-tenant isolation
- unauthorized appointment access

- invalid appointment type
- inactive doctor
- inactive hospital
- blocked slot
- appointment overlap

API VALIDATION

Verify that the complete booking path works through actual HTTP APIs, not only direct service calls.

FRONTEND VALIDATION

Connect existing patient booking UI to real APIs.

Do not use mocked frontend slots or hardcoded appointments.

The patient should be able to:

- 1 Select a hospital/doctor.
- 2 View real available slots.
- 3 Select a slot.
- 4 Submit booking.
- 5 See the actual appointment state.
- 6 Refresh and still see the persisted result.

DEPLOYMENT READINESS

Verify the integrated workflow works in the project's Docker development environment.

Do not introduce new infrastructure merely for this integration.

FINAL ARCHITECTURAL CHECK

The final dependency direction must remain:

Frontend / Voice ? API ? AI or Application Service ? Capabilities where AI is involved ? Domain Services ? Scheduling ? Integration ? EHR Connector ? Verification / Synchronization ? Events / Workflows ? Notifications / Analytics / Audit

The AI must never bypass the controlled application capabilities to directly access PostgreSQL or the EHR.

At the end, provide:

- complete booking flow implemented
- files changed
- API changes
- database changes
- integration changes
- tests executed
- failure scenario demonstrated
- remaining limitations

Prompt 26

Implement the Patient Portal foundation for CareFlow AI using the existing backend architecture, APIs, authentication, tenant boundaries, scheduling domain, appointment lifecycle, and design system. Inspect the current Patient SPA and backend before changing anything. Do not recreate working functionality and do not introduce a second API/client/auth pattern.

Build the patient-facing foundation around the real domain model:

- Patient authentication and profile must remain separate from the User authentication identity.
- Patient requests must only access the authenticated patient's own private profile, appointments, questionnaires, preferences, and permitted conversation/context data.
- Patient-facing hospital, doctor, service, appointment type, and availability data must come from real backend APIs.
- Never expose hospital-private administrative data, internal configuration, secrets, audit details, or other patients' records.
- Preserve the existing five-application frontend structure and keep Patient functionality inside the Patient SPA.

Implement the Patient SPA information architecture needed for the booking journey:

- Home / Find Care
- Hospital discovery
- Doctor discovery
- Doctor profile/details
- Appointment type selection
- Availability/calendar
- Booking review
- Appointment confirmation/status
- My Visits/Appointments
- Appointment detail
- Profile and preferences

Build reusable patient API hooks/services using the existing API client. Handle loading, empty, error, unauthorized, unavailable, and retry states consistently.

For discovery:

- Allow patients to search/filter using only supported backend fields.
- Show only hospitals that are eligible for patient-facing discovery according to the existing lifecycle rules.
- Show only active doctors and active appointment types.
- Respect hospital, department, specialty, service, consultation mode, and other supported configuration.
- Do not calculate availability in React. Availability must come from the central scheduling API.

For doctor details:

- Display only patient-safe information such as name, profile image where available, specialty, department, qualifications, experience, languages, supported consultation modes, appointment types, and other fields explicitly exposed by the API.

- Do not expose internal IDs or external-provider identifiers unless already designed as patient-safe public identifiers.

For booking:

- Reuse the deterministic scheduling engine created earlier.
- Display actual available slots returned by the backend.
- When the patient selects a slot, revalidate availability during booking.
- Handle slot becoming unavailable with a clear recovery path back to availability.
- Send an idempotency key for booking attempts and preserve it across safe client retries.
- Show appointment state based on the actual appointment lifecycle and synchronization state rather than assuming success from an HTTP response.

For appointments:

- Implement patient appointment list and detail views.
- Clearly distinguish Requested/Pending, Confirmed, Rescheduled, Completed, No-show, Cancelled, Failed, Synchronization Pending, and Reconciliation Required where those states are exposed to patients.
- Do not expose internal reconciliation mechanics unnecessarily; map operational states to safe patient-facing explanations.
- Support cancel/reschedule only through backend-authorized operations.
- Re-fetch appointment state after operations instead of assuming the mutation succeeded.

Security:

- Backend remains the security boundary.
- Add backend authorization tests proving one patient cannot read or mutate another patient's records.
- Add frontend route guards, but do not treat them as authorization.
- Do not store sensitive patient data in unsafe browser storage.
- Do not log questionnaire answers, medical details, access tokens, or private appointment payloads.

Testing:

- Backend API tests for patient ownership and appointment authorization.
- Frontend component/page tests for loading/error/empty/success states.
- End-to-end test for authenticated patient discovery ? doctor ? real availability ? booking review ? booking result.
- Verify no fake/mock slots remain in the production patient booking path.
- Run typecheck, lint, tests, and production build.

Keep the implementation aligned with the existing architecture: UI calls application APIs, application/domain services enforce business rules, scheduling owns availability, integration owns EHR communication, and the patient UI never talks directly to PostgreSQL, Redis, Celery, EHR vendors, or AI providers.

Prompt 27

Implement the Patient appointment management and questionnaire workflow on top of the existing appointment lifecycle, event bus, workflow infrastructure, authentication, and patient portal. Inspect the current implementation first and extend it without duplicating existing appointment or workflow logic.

The goal is to complete the patient-side post-booking journey: booking result ? appointment status ? questionnaire availability ? questionnaire completion ? submission ? appointment readiness information.

Use the existing appointment lifecycle and do not create a second appointment state machine.

Implement questionnaire domain support with clear ownership:

- Questionnaire definition belongs to the hospital/service/appointment configuration according to the existing model.
- Questionnaire versions must be identifiable so historical responses remain tied to the correct version.
- Questions should have stable identifiers.
- Responses belong to the authenticated patient and the relevant appointment/questionnaire version.
- Patient must not be able to submit responses for another patient's appointment.
- Hospital configuration must determine which questionnaire is applicable.
- Do not place clinical decision-making or diagnosis logic in the questionnaire workflow.

Implement the backend flow:

- 1 Appointment becomes eligible for questionnaire preparation after the appropriate booking/workflow event.
- 2 Workflow/event processing determines whether a questionnaire is required.
- 3 Patient can fetch only questionnaires assigned to their own appointment.
- 4 Patient can save progress where supported.
- 5 Patient can submit the questionnaire.
- 6 Submission is immutable or versioned after final submission according to the existing data model.
- 7 Submission creates the appropriate workflow/audit event.
- 8 Doctor/hospital workflows can consume readiness information without exposing unnecessary patient data.
- 9 Repeated event delivery must be idempotent.

Use Celery for asynchronous processing where appropriate:

- questionnaire assignment
- notification triggering
- reminder scheduling
- downstream workflow processing

Do not make the patient HTTP request depend on a Celery worker being immediately available unless the operation explicitly requires asynchronous completion. Persist the durable state first and process downstream work asynchronously.

Implement patient-facing UI:

- Appointment detail should clearly show whether questionnaire is not required, not yet available, in progress, completed, or requires action.
- Questionnaire page should support sections/questions using the configured schema.
- Support appropriate input types already defined by the domain.

- Validate required fields on both frontend and backend.
- Preserve partially completed work where supported.
- Prevent accidental duplicate final submissions.
- Show submission confirmation based on persisted backend state.

Privacy and safety:

- Questionnaire answers are sensitive patient data.
- Never write raw answers to application logs, analytics events, tracing metadata, error messages, or audit metadata.
- Audit should record that the questionnaire was accessed/submitted, not copy its contents.
- Authorization must be checked at every read/write boundary.
- Hospital/doctor users should receive only the information their role is explicitly allowed to access.

Doctor-side readiness:

- Expose a controlled appointment readiness indicator or summary through a dedicated API.
- Do not leak the entire patient questionnaire into an operational list.
- If the architecture supports authorized doctor review, provide a dedicated authorized endpoint/page for the questionnaire response rather than embedding sensitive data into general appointment lists.

Testing:

- Patient cannot access another patient's questionnaire.
- Patient cannot submit against another appointment.
- Required-field validation.
- Draft/resume behavior if supported.
- Duplicate submission/idempotency.
- Event redelivery does not create duplicate assignments or notifications.
- Appointment cancellation/rescheduling correctly affects questionnaire eligibility.
- Cross-tenant access is denied.
- Sensitive answers are absent from logs/audit payloads.
- End-to-end test: confirmed appointment ? questionnaire available ? patient completes ? submission persisted ? doctor readiness state updated.

Prompt 28

Implement the AI context, conversation persistence, capability/MCP boundary, and grounded administrative assistant foundation for CareFlow AI using the existing architecture. Inspect the current AI orchestrator, AIContext, MCP/capability layer, Redis usage, conversation models, prompt handling, and existing tests before making changes.

The AI must remain an orchestration layer, not a business-rule or data-access layer.

Establish a durable conversation model that can support:

- Patient conversations
- Doctor conversations
- Hospital Admin conversations
- Operations/Admin conversations where already supported
- Conversation messages
- Conversation metadata
- Correlation/operation identifiers where appropriate
- Context snapshots or references
- Capability executions
- Tool results/status
- Conversation ownership and tenant scope

Separate:

- Durable conversation history in PostgreSQL.
- Short-lived AI context/cache in Redis.
- Business truth in domain services/PostgreSQL.
- External truth in integration/EHR systems.

Do not put the entire durable conversation transcript into Redis as the source of truth.

Implement a structured AIContext based on the existing context design. It should be capable of carrying only the minimum state needed for the current workflow, such as:

- authenticated actor
- role
- hospital scope
- patient/doctor/hospital identifiers where authorized
- current intent
- current workflow/operation
- selected hospital
- selected doctor
- selected appointment type
- selected date/time
- appointment identifier
- current appointment state

- required missing information
- capability results
- readiness/confirmation state
- correlation/operation identifiers

Do not trust AI-generated identifiers or role/hospital claims. Resolve authorization from the authenticated backend context and validate all selected resources through domain services.

Implement the capability/MCP boundary:

- AI may request controlled capabilities such as hospital discovery, doctor discovery, availability lookup, appointment preparation, booking, appointment status, questionnaire status, and other explicitly allowed administrative/access operations.
- Each capability must have typed input/output schemas.
- Each capability must perform authorization and domain validation.
- Each capability should emit capability execution telemetry without logging sensitive payloads.
- AI must never directly execute SQL.
- AI must never call an EHR vendor SDK directly.
- AI must never bypass scheduling.
- AI must never bypass appointment lifecycle rules.
- AI must never invent availability, booking confirmation, doctor data, hospital data, or appointment status.

Implement grounded responses:

- Tool/capability results are the source for factual application state.
- The model should clearly distinguish known state from missing information.
- If no result is available, the assistant should ask for required information or report that the system cannot confirm the requested state.
- Do not fabricate citations or source references.
- Preserve structured capability outputs separately from generated natural-language responses.

Conversation persistence:

- Save user messages and assistant messages.
- Save capability execution records.
- Save relevant operation/correlation references.
- Avoid duplicating large external payloads.
- Apply retention/privacy rules already established by the project.
- Ensure conversation retrieval is tenant/user scoped.

Implement API endpoints for:

- create/list conversations
- get conversation
- send message
- retrieve relevant conversation history
- capability/tool execution status where needed

Frontend:

- Build/reuse the Patient/Doctor/Admin chat surface as appropriate.
- Streaming is optional only if it does not compromise persistence and state correctness.

- The UI must distinguish normal conversational text from structured booking/status cards.
- Never show internal tool traces or secrets to end users.
- When the assistant performs an action, show the resulting business state from the backend.

Testing:

- Cross-user and cross-hospital conversation isolation.
- Capability authorization.
- AI cannot bypass scheduling.
- AI cannot claim confirmation without verified appointment state.
- Tool failure produces safe fallback.
- Conversation persistence survives process restart.
- Redis loss does not destroy durable conversation history.
- Duplicate user requests are safely handled where the underlying operation is idempotent.
- Verify the orchestrator remains independent of vendor-specific EHR implementation and direct database access.

Prompt 29

Implement the CareFlow AI administrative/access AI booking assistant and voice-ready interaction loop using the existing AIContext, capability/MCP layer, deterministic scheduling engine, appointment workflow, EHR verification, questionnaire workflow, and conversation persistence. Do not introduce LangGraph or another agent framework unless the existing architecture explicitly requires it. Strengthen the current orchestrator instead of replacing working architecture.

The assistant must support a controlled end-to-end conversation such as: patient asks to find care ? discover eligible hospitals ? discover matching doctors/services ? present doctor cards ? user selects/explores more ? collect missing booking information ? retrieve real availability ? user selects slot ? prepare booking ? execute booking with idempotency ? verify EHR state ? synchronize ? report actual appointment state ? trigger downstream questionnaire/notification workflows.

Implement explicit orchestration states or equivalent structured context transitions. Do not rely only on free-form conversation text to determine workflow state.

The orchestrator should:

- Open each turn by loading the authenticated actor and current conversation/context.
- Validate tenant and resource scope.
- Extract intent and required entities.
- Detect missing information.
- Resolve hospital before hospital-scoped doctor/service operations where required.
- Resolve doctor and appointment type using controlled capabilities.
- Correct mismatches between user-selected resources and authoritative backend data.
- Fetch real availability only through the scheduling capability.
- Maintain a canonical booking snapshot before executing a booking operation.
- Require explicit user confirmation before final booking when the interaction design calls for confirmation.
- Generate and preserve an idempotency key for the booking operation.
- Execute booking through the appointment/application service.
- Wait for or poll the correct verification/synchronization state when the booking workflow is asynchronous.
- Report the actual resulting state rather than inferring success from tool completion.

Design structured assistant responses so the frontend can render:

- normal conversational text
- hospital cards
- doctor cards
- appointment type cards
- date/calendar selectors
- time-slot selectors
- booking review cards
- appointment status cards
- questionnaire action cards
- error/recovery actions

Keep these structures typed and versionable. Do not parse UI-critical state from arbitrary natural-language text.

“Explore more” behavior:

- If a user asks to explore more doctors, use the same search criteria and fetch the next appropriate result set.
- Do not repeat identical cards indefinitely.
- Maintain search context in AIContext/conversation state.
- Do not invent additional doctors.

Mismatch correction:

- If the user asks for a doctor/service that does not belong to the selected hospital or is inactive, do not force the request through.
- Explain the mismatch and use authoritative discovery data to offer valid next steps.
- Never allow AI-generated IDs to override backend validation.

Booking failure/recovery:

- If EHR outcome is unknown, do not tell the patient the appointment failed or succeeded prematurely.
- Show a safe pending/reconciliation state.
- Let the recovery workflow query external state before retrying.
- If the appointment is found externally, synchronize and present the resulting state.
- If no appointment is found and retry is safe, continue through the controlled retry path.
- If divergent external/internal state is detected, expose a safe “needs verification” state rather than inventing a result.

Voice readiness:

- Keep speech-to-text, text-to-speech, VAD, and browser audio concerns outside the core booking logic.
- Voice input should enter the same orchestrator path as typed input.
- Voice output should consume the same structured assistant result.
- Do not create a separate booking implementation for voice.
- If an external TTS provider is unavailable, the booking workflow must still work through text/browser speech where supported.
- Do not put audio bytes into durable conversation records unless explicitly required by the product.

Safety boundary:

- The assistant is for administrative/access workflows only.
- Do not implement diagnosis, prescribing, treatment recommendations, clinical assessment, or interpretation of medical symptoms.
- When a user asks for clinical advice, route to an appropriate safe informational boundary rather than executing a clinical workflow.

Testing:

- Multi-turn booking conversation.
- Hospital-first resolution.
- Doctor discovery and “explore more”.
- Appointment type selection.
- Real availability selection.
- Explicit confirmation.
- Successful booking.

- EHR timeout/unknown outcome.
- Recovery after unknown outcome.
- Duplicate user booking request.
- Cross-tenant attempt.
- Invalid/inactive doctor.
- Invalid appointment type.
- Slot collision.
- Conversation restart/resume.
- Typed input and voice input follow the same business path.

Prompt 30

Implement the integrated CareFlow AI workflow verification, observability, failure demonstration, and production-readiness pass across the full system. Treat this as an architecture-conformance implementation task, not a broad rewrite. Inspect all existing work from Prompt1–Prompt29 before changing anything and preserve working implementations.

Validate and harden the complete product flow:

Patient: authentication ? patient profile ? hospital discovery ? doctor discovery ? appointment type ? real availability ? booking review ? appointment booking ? EHR create ? external verification ? synchronization ? confirmed/appropriate appointment state ? questionnaire workflow ? notifications/status.

Doctor: authentication ? own profile ? availability/calendar ? appointment visibility ? authorized questionnaire/readiness information ? appointment lifecycle actions.

Hospital Admin: hospital configuration ? departments/specialties/services ? appointment types ? doctors ? calendars/availability ? operational visibility ? authorized appointment/questionnaire information.

Operations/Admin: only the operational responsibilities already defined by the architecture; do not expand privileges simply for convenience.

Platform Admin: hospital onboarding/review/lifecycle and platform-level responsibilities, without exposing tenant-private data unnecessarily.

Perform a systematic architecture audit:

- Frontends communicate through backend APIs.
- Backend routers do not contain domain logic that belongs in services.
- Domain services own business rules.
- Scheduling remains deterministic and independent of AI.
- AI orchestrator uses capabilities/application services rather than SQL or vendor SDKs.
- EHR vendor details stay behind the connector/integration layer.
- Verification is separate from external request execution.
- Synchronization is explicit.
- Celery handles asynchronous work without becoming the source of truth.
- PostgreSQL remains the durable source of truth for internal state.
- Redis remains appropriate for short-lived context/cache/broker duties.
- Hospital tenant isolation is enforced server-side.
- Frontend route guards are not treated as authorization.
- Sensitive information is not logged.

Implement end-to-end traceability:

- Generate/propagate correlation ID for every request.
- Generate operation IDs for meaningful business operations.
- Link appointment operations to scheduling decisions.
- Link appointment operations to integration operations.
- Link integration operations to verification results.
- Link synchronization/reconciliation operations back to the originating appointment.

- Link workflow executions/events to their source operation.
- Link AI capability executions to the conversation/request where appropriate.

Create safe operational records for:

- API request outcome
- capability execution
- scheduling attempt/result
- appointment operation
- integration operation
- verification result
- synchronization result
- reconciliation record
- workflow execution
- notification execution
- AI usage/evaluation event
- audit event

Do not log:

- passwords
- JWTs
- API keys
- EHR credentials
- raw questionnaire answers
- full medical information
- full private conversation payloads
- audio contents unless explicitly required and protected

Implement the required failure demonstration for the product:

- Use the Mock EHR to deterministically simulate an unknown external outcome, such as timeout after external creation but before internal confirmation.
- Show the internal appointment remaining in an appropriate pending/recovery state.
- Query external state before retry.
- Demonstrate recovery when the external appointment exists.
- Demonstrate the safe-not-found retry path when it does not exist.
- Demonstrate reconciliation-required behavior for divergent state.
- Ensure duplicate external appointments are not created by the recovery process.
- Preserve correlation and operation IDs throughout the failure/recovery chain.

Build a test matrix covering at minimum:

- 1 authentication/authorization
- 2 cross-hospital isolation
- 3 patient ownership
- 4 doctor ownership
- 5 hospital lifecycle gates
- 6 doctor lifecycle gates

7	appointment type compatibility
8	working hours
9	blocked/leave periods
10	appointment overlap
11	concurrent booking
12	booking idempotency
13	appointment cancellation
14	appointment rescheduling
15	EHR success
16	EHR validation failure
17	EHR timeout/unknown outcome
18	verification mismatch
19	synchronization
20	reconciliation
21	questionnaire assignment
22	questionnaire submission
23	workflow idempotency
24	notification failure/retry
25	AI capability authorization
26	AI grounded response behavior
27	AI booking confirmation correctness
28	conversation persistence
29	Redis unavailability
30	Celery worker unavailability
31	API restart/recovery
32	Docker Compose startup
33	frontend production builds
34	frontend/backend contract consistency

Add contract/integration tests where appropriate so that frontend and backend drift is detected.

Run a final repository cleanup:

- Remove dead duplicate implementations.
- Remove fake booking data from production paths.
- Remove obsolete vendor-specific AI/TTS assumptions.
- Remove unused imports and stale environment variables.
- Ensure .env.example contains required configuration names without secrets.
- Ensure migrations are reproducible.
- Ensure Docker startup is deterministic and non-destructive.
- Ensure health/readiness endpoints reflect real dependencies appropriately.
- Ensure no service silently falls back to insecure behavior in production.

Create or update architecture documentation that explains the implemented request flow and boundaries: Patient UI ? API ? Auth/Tenant Context ? AI/Application layer where applicable ? Capability/MCP ? Domain Services ?

Scheduling/Appointment ? Integration ? Verification ? Synchronization ? Workflow/Event Bus ?
Notifications/Questionnaire/Analytics.

Document the failure recovery path and the meaning of the major appointment and operational states.

Finally, execute the relevant backend tests, frontend tests, typecheck, lint, migration validation, Docker build/start checks, and production builds. Fix implementation defects found by these checks rather than merely documenting them. Do not claim the system is production-ready unless the repository actually passes the implemented verification checks. Produce a concise final implementation report listing:

- what was verified
- what was fixed
- tests/builds executed
- any remaining known limitations
- exact commands used for verification

Prompt 31

Implement the CareFlow AI notification and reminder subsystem using the existing appointment lifecycle, workflow/event bus, Celery infrastructure, patient/doctor roles, hospital communication configuration, audit/observability model, and tenant isolation. Inspect the existing notification code before creating anything new. Reuse existing abstractions and do not introduce a second asynchronous job system.

Build notifications as a downstream workflow capability, not as part of the core appointment transaction.

Support the notification events already implied by the product workflow, including where applicable:

- appointment requested
- appointment pending/synchronization pending
- appointment confirmed
- appointment rescheduled
- appointment cancelled
- appointment completed
- questionnaire available
- questionnaire reminder
- appointment reminder
- reconciliation/recovery status requiring patient-safe action
- operational failures that require an authorized staff/admin notification

Define a notification domain model that can track:

- notification ID
- recipient/user
- hospital scope where applicable
- appointment/reference entity
- event/type
- channel
- template/version
- status
- scheduled time
- sent time
- provider/message identifier where applicable
- retry count
- correlation ID
- operation ID
- created/updated timestamps

Use a provider abstraction so notification delivery is not hard-coded to one vendor:

- email provider interface
- SMS provider interface where configured
- in-app notification interface

- optional push provider boundary if already present

For the MVP, provide a safe development/mock provider if real providers are not configured. Never silently send real messages using test credentials.

Implement event-driven notification triggering:

- 1 Core business operation persists durable state.
- 2 Appropriate domain/application event is emitted.
- 3 Notification workflow consumes the event.
- 4 Recipient and communication preferences are resolved.
- 5 Template is selected.
- 6 Notification is persisted before/around delivery according to the chosen reliability pattern.
- 7 Provider is called asynchronously.
- 8 Delivery result is recorded.
- 9 Transient failures are retried safely.
- 10 Permanent failures are marked failed without infinite retries.

Make event handling idempotent:

- Duplicate appointment events must not create duplicate notifications.
- Use an event/message identity or equivalent deduplication key.
- Notification retries must not create multiple logical deliveries unintentionally.
- Preserve the original correlation/operation identity.

Respect hospital communication configuration:

- hospital-level channel enablement
- supported notification types
- reminder configuration where already modeled
- patient communication preferences
- doctor/admin communication rules
- timezone-aware scheduling

Never allow a patient preference to override a mandatory security/operational notification where the existing product rules define it as mandatory.

Implement appointment reminders:

- Derive reminder timing from persisted appointment data and configured rules.
- Use the appointment's effective timezone.
- Do not schedule reminders from stale frontend data.
- If an appointment is cancelled/rescheduled, obsolete reminders must be cancelled or rendered ineffective.
- Rescheduling must create the correct new reminder behavior without duplicating old reminders.

Use Celery/Beat only for asynchronous execution:

- scheduled reminder dispatch
- retrying transient delivery failures
- cleanup/reconciliation of notification jobs
- periodic checks where required

Do not make booking depend on successful notification delivery.

Patient UI:

- Add an in-app notification center if the existing Patient SPA structure supports it.
- Show safe notification summaries and appointment actions.
- Never expose internal errors, provider responses, credentials, or operational logs.
- Deep-link notification actions to the relevant appointment/questionnaire page.
- Keep notification state synchronized with backend state.

Doctor/Hospital Admin UI:

- Show only notifications relevant to their role and hospital.
- Do not leak patient-private content into broad operational notifications.
- Use role-specific summaries.

Security/privacy:

- Tenant isolation applies to notification queries.
- Recipient authorization must be checked.
- Sensitive patient information should be minimized in notification payloads.
- Never log email/SMS contents containing sensitive data.
- Do not put secrets or provider credentials in notification records.

Testing:

- event creates one logical notification
- duplicate event is idempotent
- transient provider failure retries
- permanent provider failure stops retrying
- cancelled appointment prevents stale reminder delivery
- rescheduled appointment updates reminder behavior
- timezone correctness
- patient notification isolation
- hospital isolation
- provider abstraction
- Celery worker unavailable
- notification failure does not fail booking
- end-to-end confirmed appointment ? notification workflow

Run backend tests, frontend tests/typecheck/build, and relevant Celery worker checks. Document notification states and retry behavior.

Prompt 32

Implement the Doctor operational workflow in CareFlow AI using the existing Doctor domain, calendar/availability model, appointment lifecycle, questionnaire workflow, authorization, tenant isolation, notification subsystem, and frontend Doctor SPA. Inspect existing doctor functionality first and extend it without duplicating business rules.

The Doctor application should provide a controlled operational workspace centered on the doctor's own hospital-scoped data.

Implement the Doctor dashboard with real backend data:

- today's appointments
- upcoming appointments
- appointment status summary
- pending/relevant questionnaire readiness indicators
- availability/calendar summary
- notifications/tasks where already supported

Do not calculate appointment counts or status from frontend assumptions. Query the backend.

Doctor authorization:

- A doctor may access only their own doctor record and authorized operational resources.
- A doctor must not access another doctor's private operational data.
- Doctor hospital scope must be enforced server-side.
- Never trust doctor_id, hospital_id, or role values supplied by the frontend.
- Backend authorization remains the security boundary.

Implement appointment management for doctors:

- list appointments for the authenticated doctor
- filter by date/status where supported
- view appointment details
- update only states/actions explicitly allowed by the domain
- complete appointment
- mark no-show where authorized
- cancel where policy permits
- reschedule through the central scheduling/application service where permitted

Do not allow doctors to directly mutate appointment status through arbitrary status strings.

Use explicit service methods and state transitions.

Appointment detail should expose only the information appropriate to the doctor role:

- patient identity fields allowed by the product
- appointment type
- date/time/timezone
- consultation mode
- relevant status
- questionnaire readiness

- authorized questionnaire information
- operational notes if explicitly supported

Do not expose unrelated hospital configuration or another patient's records.

Integrate questionnaire readiness:

- Show whether the patient has a required questionnaire.
- Show whether it is pending, in progress, or submitted.
- If doctor review is supported, provide a dedicated authorized response view.
- Do not embed raw questionnaire answers in appointment list APIs.
- Do not expose questionnaire data for appointments outside the doctor's scope.

Calendar integration:

- Doctor should be able to view their calendar and availability.
- Reuse the existing calendar/availability APIs.
- Availability changes must go through the domain service.
- Prevent changes that violate existing appointments or hospital configuration rules.
- Respect timezone and working-hour boundaries.

Doctor profile:

- View/edit only fields the doctor is authorized to manage.
- Hospital-controlled fields such as hospital assignment, department assignment, lifecycle state, and externally managed identifiers must remain protected.
- Do not allow a doctor to self-activate, self-assign to another hospital, or alter administrative ownership.

Notifications:

- Show relevant appointment/questionnaire/system notifications.
- Link notification actions to actual backend resources.
- Do not expose hospital-wide administrative notifications unless authorized.

Frontend:

- Doctor navigation should remain coherent and focused:

Overview Appointments Calendar Availability Profile Notifications

- Reuse existing React Router, API client, UI components, and authentication.
- Add proper loading, empty, error, retry, and permission states.
- Avoid fake/static appointment data in production paths.

Testing:

- doctor can access own appointments
- doctor cannot access another doctor's appointments
- doctor cannot access another hospital
- doctor cannot modify protected doctor fields
- appointment state transitions are validated
- cancellation/rescheduling uses scheduling service
- questionnaire access is correctly scoped
- availability changes preserve existing booking constraints
- frontend route guards and backend authorization

- production typecheck/build

Verify the Doctor workflow against the full architecture rather than adding isolated UI behavior.

Prompt 33

Implement the Hospital Admin operational dashboard and hospital-level operational management using the existing hospital lifecycle, tenant isolation, doctor management, service configuration, appointment type configuration, scheduling, appointment, questionnaire, notification, audit, and integration layers.

Inspect the current Hospital Admin SPA and backend before implementation. Reuse existing APIs and domain services. Do not move business rules into React.

Build the Hospital Admin dashboard around the authenticated hospital:

- hospital lifecycle/status
- doctor count/status
- department/specialty/service summary
- appointment operational summary
- upcoming appointments
- pending operational actions
- questionnaire/workflow indicators
- integration health/status where authorized
- recent audit/operational activity where appropriate

Every query must be hospital-scoped server-side.

Hospital Admin must not:

- approve its own hospital
- access another hospital
- change platform-level configuration
- access raw credentials/secrets
- bypass doctor lifecycle rules
- bypass scheduling
- directly manipulate EHR records
- directly modify appointment state outside allowed service operations

Implement hospital operational views:

- 1 Doctors
- 2 Departments
- 3 Specialties
- 4 Services
- 5 Appointment Types
- 6 Calendars/Availability
- 7 Appointments
- 8 Questionnaires
- 9 Communication
- 10 Integrations
- 11 Audit/Operations where authorized

Use the existing CRUD and lifecycle services rather than implementing duplicate endpoints.

Appointment operations:

- View hospital-scoped appointments.
- Filter by date, doctor, department, appointment type, consultation mode, and status where supported.
- Use real appointment state from the backend.
- Provide operational actions only when allowed by policy.
- Rescheduling must use the scheduling/application service.
- Cancellation must use the appointment service.
- Do not provide a generic "set status" endpoint from the UI.

Doctor operations:

- Invite/create doctor according to existing lifecycle.
- Assign supported department/specialty/service relationships.
- Activate/deactivate/suspend/reinstate through explicit lifecycle methods.
- Ensure approved/active hospital requirements remain enforced.

Calendar/availability:

- Provide hospital admin visibility into doctor availability where authorized.
- Allow configuration changes only within the supported domain.
- Do not directly edit generated appointment slots.
- Do not let hospital admins create availability that violates doctor/hospital lifecycle constraints.

Questionnaires:

- Manage configured questionnaire definitions and versions where authorized.
- View operational completion/readiness summaries.
- Protect patient questionnaire responses.
- Do not place raw sensitive answers into dashboard aggregation endpoints.

Communication:

- Reuse the notification configuration created previously.
- Allow hospital admins to configure supported channels/types/timing.
- Validate all changes server-side.
- Record configuration changes in audit.

Integration:

- Show connector status and safe health information.
- Never display secrets.
- Do not allow arbitrary external API URLs from ordinary configuration.
- Integration operations remain inside the integration layer.

Audit/operations:

- Show safe operational events for the hospital.
- Include timestamp, actor, action, resource type, success/failure, correlation/operation ID where appropriate.
- Never display raw sensitive payloads.

Performance:

- Dashboard APIs should use aggregated queries or dedicated read models/endpoints where necessary.

- Avoid N+1 queries.
- Add pagination to large lists.
- Add server-side filtering.
- Do not load all hospital appointments/doctors into the browser.

Security tests:

- Hospital Admin A cannot access Hospital B.
- Hospital Admin cannot approve own hospital.
- Hospital Admin cannot change platform-only resources.
- Hospital Admin cannot read another patient's private data.
- Hospital Admin cannot bypass appointment/scheduling state transitions.
- Hospital Admin cannot retrieve integration secrets.

Frontend:

- Use coherent navigation and reusable tables/forms.
- Add confirmation for destructive/high-impact actions.
- Show actual backend state.
- Handle loading, empty, errors, permission denied, and stale data.
- Maintain responsive usability.

Testing:

- backend tenant/role tests
- lifecycle tests
- appointment authorization
- doctor authorization
- configuration authorization
- integration secret protection
- frontend typecheck/build
- end-to-end hospital admin operational workflow

Prompt 34

Implement the Platform Admin and Operations/Admin control surfaces required by the existing CareFlow AI architecture. Inspect the current roles, backend authorization, hospital lifecycle, audit/observability, integration operations, workflow/reconciliation, and frontend applications before making changes.

Do not expand administrative privileges merely because an endpoint is convenient. Every capability must have an explicit role boundary and resource scope.

Platform Admin responsibilities should center on platform-wide governance:

- hospital application review
- approve/reject hospital applications
- request corrections/resubmission where supported
- suspend/reinstate hospitals where supported
- view platform-level hospital status
- view safe platform operational metrics
- manage platform-level configuration only where explicitly defined

Hospital approval:

- Platform Admin can review submitted hospital information.
- Approval transitions must use explicit lifecycle methods.
- Rejection must support a structured reason where the existing domain requires it.
- Hospital Admin must never approve its own hospital.
- Approval actions must create audit events.
- Approval must not directly activate unrelated doctors or create appointments unless the domain workflow explicitly does so.

Platform-level views must avoid exposing unnecessary tenant-private information.

Implement Operations/Admin capabilities for operational recovery where the architecture supports this role:

- view failed integration operations
- view verification failures
- view reconciliation-required records
- inspect workflow execution failures
- inspect notification failures
- retry supported transient/recoverable operations
- resolve reconciliation records through controlled actions
- view correlation/operation traces
- monitor Celery/workflow execution health where exposed

Operations actions must be explicit:

- retry integration operation
- retry notification
- retry workflow
- mark reconciliation as resolved with reason

- escalate when required

Do not create a generic “force success” operation.

For EHR recovery:

- Operations user must see enough metadata to identify the operation.
- Do not expose credentials or sensitive payloads.
- Show external operation identifier where safe.
- Show verification state.
- Show retry eligibility.
- Show last error category/message sanitized for operational use.
- Show retry count.
- Show correlation and operation identifiers.

Authorization:

- Platform Admin has platform-wide scope.
- Operations/Admin has only the operational capabilities explicitly defined.
- Hospital Admin remains hospital-scoped.
- Doctor remains doctor-scoped.
- Patient remains patient-scoped.
- Enforce all permissions on the backend.
- Do not rely on frontend route visibility.

Audit: Every high-impact administrative action should record:

- actor
- role
- hospital/resource scope
- action
- resource
- previous state where safe
- resulting state
- correlation ID
- operation ID
- timestamp
- success/failure
- safe reason metadata

Do not record:

- passwords
- tokens
- credentials
- raw patient questionnaire answers
- complete medical data
- full private conversation contents

Frontend: Create or extend the appropriate Platform Admin and Operations/Admin SPA surfaces using the existing frontend architecture.

Platform Admin navigation may include:

- Overview
- Hospitals
- Hospital Review
- Platform Operations
- Audit

Operations navigation may include:

- Operations Overview
- Integration Failures
- Reconciliation
- Workflow Failures
- Notifications
- Traces/Audit

Use tables with:

- pagination
- server-side filters
- status indicators
- timestamps
- correlation/operation IDs
- safe error summaries
- explicit action controls

Do not expose internal stack traces to users.

Testing:

- platform admin can review hospitals
- hospital admin cannot access platform review
- operations user cannot perform platform-only approval
- hospital admin cannot access global reconciliation
- patient/doctor cannot access operations endpoints
- retry authorization is enforced
- reconciliation resolution is audited
- cross-tenant data exposure is prevented
- frontend route guards
- backend authorization
- production builds/typechecks

Prompt 35

Implement the final product-wide analytics, AI usage/evaluation, audit reporting, and performance/readiness layer for CareFlow AI using the existing architecture. Inspect the current analytics, audit, AI usage, appointment, scheduling, workflow, notification, and observability implementations before changing anything.

Analytics must be derived from authoritative persisted business events/state. Do not build analytics by parsing frontend screens or AI-generated text.

Define safe analytics boundaries.

Product/operational metrics may include:

- hospital onboarding counts by lifecycle state
- active doctors
- configured services/appointment types
- appointment requests
- booking outcomes
- confirmed appointments
- cancellations
- reschedules
- completed/no-show counts
- questionnaire completion rates
- notification delivery outcomes
- EHR integration success/failure counts
- verification outcomes
- reconciliation counts
- workflow success/failure counts
- AI capability execution counts
- AI response/operation latency
- error rates

Do not expose patient-sensitive information through aggregate analytics.

Implement analytics dimensions carefully:

- time period
- hospital where authorized
- doctor where authorized
- appointment type
- consultation mode
- operation type
- outcome/status

Tenant isolation:

- Hospital Admin sees only hospital-level analytics.
- Doctor sees only metrics relevant to their own operational scope.

- Platform Admin can see platform-wide aggregate metrics.
- Operations/Admin sees operational reliability metrics according to authorization.
- Patient sees only their own appointment/activity information, not global analytics.

Use server-side aggregation.

- Avoid loading raw appointment/event rows into the frontend.
- Add pagination for detailed operational event lists.
- Add appropriate database indexes.
- Consider materialized/derived metrics only when justified by scale.
- Do not introduce premature analytics infrastructure if PostgreSQL aggregation is sufficient for the current product.

Implement AI usage tracking:

- model/provider identifier
- request category
- capability/tool usage
- latency
- token/usage information where available
- success/failure
- fallback usage
- correlation/operation ID
- conversation reference
- timestamp

Never store sensitive prompts/responses in usage metrics unless explicitly required and protected.

Implement AI evaluation hooks for the product's administrative assistant:

- groundedness/grounded-response checks where supported
- capability success
- booking-state correctness
- hallucination/error incidents where detected
- tool-selection failures
- missing-information handling
- recovery-path correctness
- latency
- failure rate

Do not treat an automatically generated evaluation score as a clinical or business truth. Store evaluation metadata for engineering/product analysis.

Implement operational dashboards/endpoints:

- booking reliability
- EHR reliability
- verification/reconciliation health
- workflow failures
- notification delivery
- AI assistant health

- API latency/error summaries

Add useful time-window filters and safe tenant scoping.

Performance hardening:

- Inspect slow queries.
- Add indexes based on actual query patterns.
- Avoid N+1 queries.
- Paginate large datasets.
- Avoid returning unnecessary columns.
- Avoid synchronous external HTTP inside long database transactions.
- Ensure Celery tasks do not hold database sessions longer than necessary.
- Ensure Redis is used appropriately for short-lived state rather than durable business truth.
- Ensure external calls have explicit timeout behavior.
- Ensure retries use bounded backoff.
- Ensure idempotency exists for retried business operations.

Health/readiness:

- `/health`` should represent process liveness.
- readiness should verify the dependencies genuinely required to serve traffic.
- Do not make health checks perform destructive or expensive operations.
- Include safe dependency status for PostgreSQL/Redis/worker/integration where appropriate.
- Never expose credentials or detailed infrastructure secrets.

Security/readiness audit:

- Search for hard-coded secrets.
- Search for unsafe logging.
- Search for arbitrary SQL in API routes.
- Search for direct EHR calls outside integration.
- Search for direct Redis use where domain persistence is required.
- Search for client-controlled `hospital_id/doctor_id/patient_id` trust.
- Search for generic status mutation endpoints.
- Search for fake availability/booking data.
- Search for production-only bypasses.
- Search for disabled authorization dependencies.
- Search for unbounded retries.
- Search for missing idempotency on external operations.

Run a full verification suite:

- backend unit tests
- backend integration tests
- authorization/tenant tests
- scheduling concurrency tests
- EHR verification/recovery tests
- questionnaire/workflow tests
- notification tests

- AI capability/orchestrator tests
- frontend tests
- frontend typecheck
- frontend production builds
- Alembic migration validation
- Docker build
- Docker Compose startup
- health/readiness checks
- Celery worker/Beat startup
- representative end-to-end booking flow

Perform final documentation updates:

- architecture boundaries
- role matrix
- booking lifecycle
- EHR verification/recovery
- questionnaire workflow
- notification workflow
- AI capability boundary
- tenant isolation
- observability/correlation IDs
- analytics/AI usage
- local development and deployment commands

Do not claim full production readiness merely because builds pass. Report actual verification results and remaining limitations. Remove duplicate/obsolete code only when confirmed unused. Preserve working functionality and avoid broad rewrites.

The final result should leave one coherent CareFlow AI architecture in which: Patient/Doctor/Admin UIs ? backend APIs ? authentication/tenant context ? application/domain services ? deterministic scheduling/appointment ? integration/EHR ? verification/synchronization ? event/workflow system ? questionnaire/notifications/analytics/audit, with AI acting only through controlled capabilities and never bypassing business or safety boundaries.

Prompt 36

Implement the CareFlow AI API contract hardening and frontend/backend integration layer across all five applications: Patient, Doctor, Hospital Admin, Operations Admin, and Platform Admin. Inspect the existing routers, Pydantic schemas, TypeScript API clients, generated/handwritten types, authentication handling, error handling, and current frontend calls before making changes.

The objective is to make the backend and all frontends consume one coherent contract without duplicating business logic.

First inventory every existing API used by the frontends:

- authentication
- patient profile
- hospital discovery/configuration
- doctor discovery/profile
- departments/specialties/services
- appointment types
- calendars
- availability
- appointments
- questionnaires
- notifications
- AI conversations
- capabilities/tool execution where exposed
- hospital administration
- platform administration
- operations/reconciliation
- analytics/audit
- health/readiness

For every endpoint verify:

- HTTP method
- path
- request schema
- response schema
- authentication requirement
- role requirement
- hospital/resource scope
- error response format
- pagination format
- filtering/sorting behavior
- timestamp/timezone representation
- idempotency requirements

- correlation ID behavior

Create consistent backend API conventions:

- stable response models
- consistent validation errors
- consistent authorization errors
- consistent not-found behavior
- consistent conflict behavior
- consistent business-rule errors
- consistent pagination metadata
- consistent timestamps
- consistent correlation/operation identifiers

Do not expose SQLAlchemy models directly from API routes.

Use explicit Pydantic request/response schemas.

For mutation endpoints, distinguish:

- validation failure
- unauthorized
- forbidden
- not found
- conflict
- business-rule rejection
- transient dependency failure
- asynchronous/pending operation

Do not collapse all failures into HTTP 500.

For asynchronous operations:

- Return a clear persisted operation/appointment state.
- Provide a status endpoint where required.
- Frontend must poll or refresh based on actual state rather than assuming completion.

Idempotency:

- Identify every externally meaningful mutation.
- Booking, cancellation, rescheduling, EHR operations, notification-triggering actions, and other retryable operations must have appropriate idempotency.
- Document which endpoints require an Idempotency-Key.
- Ensure repeated requests return the same logical result where appropriate.

Pagination:

- Add server-side pagination to all potentially large list endpoints.
- Do not load all appointments, doctors, notifications, audit records, reconciliation records, or conversations into the browser.
- Return stable pagination metadata.
- Preserve filters while moving between pages.

Time handling:

- Use timezone-aware datetime values.
- Establish one documented API representation.
- Never mix naive and timezone-aware datetimes.
- Ensure frontend rendering converts timestamps for display without changing business meaning.
- Appointment/availability APIs must preserve the relevant hospital/doctor timezone.

Frontend API layer:

- Centralize API access through the existing API client.
- Remove duplicated fetch/axios configuration.
- Centralize auth token handling.
- Centralize correlation ID propagation where applicable.
- Normalize API errors into typed frontend errors.
- Do not put authorization logic only in API helpers; backend remains authoritative.

For each SPA:

- Replace fake/static API data in production paths.
- Add typed request/response models.
- Add loading/error/empty states.
- Handle 401 by the existing authentication flow.
- Handle 403 with a clear permission state.
- Handle 409 as a business conflict requiring user recovery.
- Handle pending/async states explicitly.
- Prevent duplicate mutation requests from rapid clicks.

AI endpoints:

- Keep structured capability results separate from generated text.
- Never allow frontend-generated AI state to become authoritative.
- Booking cards must contain backend-issued identifiers and current state.
- Time slots shown in the AI UI must originate from the scheduling API/capability.
- Appointment confirmation cards must use verified appointment state.

Test contract consistency:

- Backend schema tests.
- Frontend API client tests.
- Authentication error tests.
- 403/409 handling.
- Pagination tests.
- Idempotency tests.
- Timezone tests.
- Appointment async-state tests.
- Representative end-to-end flows for every SPA.

If OpenAPI is already used, make it the canonical API contract and update it from the actual backend schemas. Do not maintain a manually duplicated contract that can drift.

Run:

- backend tests

- frontend typecheck
- frontend tests
- all production builds
- representative API integration tests

Fix contract drift found during implementation rather than merely documenting it.

Prompt 37

Implement the CareFlow AI real-time booking state and frontend synchronization layer so that Patient, Doctor, Hospital Admin, Operations Admin, and AI surfaces always reflect authoritative backend state after mutations and asynchronous workflows.

Inspect the current frontend state management, React Query/custom hooks if present, API client, appointment APIs, Celery/workflow status APIs, notification system, and AI conversation state before changing anything. Reuse the existing state architecture.

The core rule is: the frontend is a projection of backend state, never the source of truth.

Implement a coherent synchronization strategy for:

- appointment booking
- cancellation
- rescheduling
- EHR synchronization
- verification
- questionnaire assignment/submission
- notification delivery/status
- reconciliation
- doctor availability
- hospital lifecycle
- doctor lifecycle

After every mutation:

- 1 Submit the operation.
- 2 Record the returned operation/resource state.
- 3 Update local UI optimistically only where safe.
- 4 Revalidate authoritative backend state.
- 5 Reflect the final persisted state.
- 6 Handle pending/recovery states explicitly.

Do not show “confirmed” simply because the POST request returned 200/201.

For booking:

- When booking starts, display a controlled pending state.
- Preserve the selected slot and booking operation identifier.
- If the backend returns synchronization pending, show that state.
- Poll or subscribe to the appointment status endpoint where appropriate.
- Stop polling after terminal state.
- Use bounded polling intervals.
- Do not create multiple booking requests because the user refreshes the page.
- Persist the client-side operation reference safely enough to recover UI state after navigation.

For unknown EHR outcome:

- Display a safe pending/recovery state.
- Do not automatically create another appointment from the browser.
- Let the backend recovery process resolve the state.
- Refresh appointment state after recovery.
- If reconciliation is required, show a safe message and appropriate next action.
- Never expose internal retry details to patients unless explicitly designed.

Cancellation:

- Require the current appointment state from the backend.
- Submit cancellation through the appointment service.
- Refresh appointment detail/list.
- Remove or update stale reminders/notifications through backend workflow.
- Do not let the UI display cancelled while the backend remains confirmed/pending.

Rescheduling:

- Fetch current availability from the scheduling API.
- Do not reuse stale slots from a previous query.
- Submit through the backend scheduling/appointment service.
- Refresh appointment and availability.
- Ensure the old slot and new slot are reflected correctly.

Availability synchronization:

- Cache only within a controlled short lifetime.
- Invalidate availability after booking/cancellation/rescheduling.
- Prevent stale slot selection.
- Backend must revalidate the slot regardless of frontend cache.

Doctor dashboard:

- Refresh appointment lists after mutations.
- Reflect completed/no-show/cancelled/rescheduled states from backend.
- Do not mutate list rows independently in ways that can diverge from server state.

Hospital Admin:

- Refresh doctor lifecycle/configuration lists after changes.
- Refresh operational appointment summaries after appointment mutations.
- Do not calculate hospital metrics solely from stale client state.

AI chat:

- When a capability executes a mutation, the assistant result should reference the operation/resource state.
- After booking, refresh the structured appointment state.
- If the operation remains pending, keep the conversation state pending rather than generating a final confirmation.
- Resume a pending operation safely when the user returns to the conversation.

Implement reusable synchronization helpers/hooks where useful:

- invalidate appointment
- invalidate availability

- invalidate doctor
- invalidate hospital
- invalidate questionnaire
- invalidate notification
- invalidate conversation
- poll operation status
- refresh after mutation

Avoid global refetch storms.

Handle browser lifecycle:

- page refresh
- route change
- browser back/forward
- duplicate tabs
- network reconnect
- expired authentication
- temporary backend failure

Use safe retry rules:

- GET/read requests may retry according to existing client policy.
- Mutations must not be blindly retried unless idempotency is guaranteed.
- EHR operations are never retried from the browser.

Testing:

- booking pending ? confirmed
- booking pending ? reconciliation
- booking pending ? failed
- cancellation synchronization
- rescheduling synchronization
- questionnaire assignment refresh
- duplicate click protection
- page refresh during pending booking
- browser reconnect
- stale availability
- two tabs attempting the same slot
- AI booking state synchronization
- all five SPAs build and typecheck successfully.

Prompt 38

Implement comprehensive security hardening for CareFlow AI across authentication, authorization, tenant isolation, API validation, frontend security, secrets handling, file/document handling where present, AI capabilities, integrations, and operational interfaces. Inspect the existing implementation and fix concrete security gaps without redesigning unrelated working features.

Treat PostgreSQL, authenticated backend context, and explicit domain authorization as authoritative.

Authentication:

- Verify password hashing uses Argon2 or the already approved secure password mechanism.
- Ensure passwords are never logged or returned.
- Validate JWT signature, expiry, issuer/audience where configured, and token type.
- Ensure refresh-token behavior is secure and consistent with the existing design.
- Reject malformed/expired/revoked credentials according to the current architecture.
- Avoid putting sensitive tokens into URLs.
- Review browser storage for access/refresh tokens and use the safest existing architecture supported by the project.

Authorization:

- Centralize role checks.
- Centralize resource ownership checks.
- Derive user/hospital/doctor/patient scope from authenticated backend context and database state.
- Never trust client-supplied role, hospital_id, patient_id, doctor_id, or ownership flags.
- Enforce authorization on every read and mutation endpoint.
- Ensure service-layer authorization cannot be bypassed by another router.
- Frontend guards are only UX, not security.

Tenant isolation:

- Test every hospital-scoped resource.
- Test forged hospital identifiers.
- Test path/query/body identifier substitution.
- Test cross-tenant appointment access.
- Test cross-tenant doctor access.
- Test cross-tenant questionnaire access.
- Test cross-tenant notification access.
- Test cross-tenant conversation/capability access.
- Test cross-tenant audit/operations access.

API validation:

- Reject unexpected or invalid values.
- Validate enum/state transitions server-side.
- Prevent mass assignment by using explicit schemas.
- Do not accept arbitrary model fields from clients.

- Enforce maximum page sizes.
- Validate date ranges.
- Validate appointment duration against configured appointment type.
- Validate pagination/filter inputs.
- Validate idempotency keys.

SQL/data layer:

- Search for raw SQL that interpolates user input.
- Ensure parameterized queries.
- Prevent unsafe dynamic ordering/filter construction.
- Review transaction boundaries.
- Prevent accidental unscoped queries in hospital-sensitive services.

External integrations:

- Store credentials only through the configured secret mechanism.
- Never return secrets from APIs.
- Never log Authorization headers.
- Validate external responses before updating internal state.
- Set connection/read timeouts.
- Use bounded retries only for safe transient operations.
- Do not follow arbitrary user-provided external URLs.

AI/MCP security:

- Capability schemas must validate inputs.
- Capability execution must resolve authorization independently of the model.
- Model output must not be treated as trusted instructions.
- Prevent prompt content from changing role/tenant scope.
- Do not expose internal tool schemas unnecessarily to end users.
- Do not allow AI to invoke arbitrary capabilities outside the approved allowlist.
- Prevent direct SQL/database access from the AI layer.
- Prevent direct vendor/EHR access from the AI layer.
- Prevent AI from bypassing scheduling and appointment state transitions.
- Ensure generated identifiers are validated against authoritative records.

Prompt injection/data exfiltration:

- Treat retrieved/document/user-provided text as untrusted.
- Do not allow text content to override system/application authorization.
- Do not allow one patient's private content to enter another patient's context.
- Scope conversation retrieval by user and tenant.
- Avoid sending unnecessary private data to LLM providers.
- Minimize prompts/context to required information.
- Track provider/model configuration without logging sensitive prompt payloads.

File/document handling where present:

- Validate file type and size.

- Prevent path traversal.
- Store uploads outside executable/static directories.
- Generate server-side storage identifiers.
- Do not trust original filenames.
- Apply tenant/owner authorization to document access.
- If OCR/text extraction exists, treat extracted content as untrusted input to AI.

Frontend security:

- Avoid unsafe HTML rendering.
- Sanitize or safely render markdown if used.
- Never interpolate untrusted HTML.
- Avoid secrets in Vite/public environment variables.
- Never put backend private credentials into frontend environment variables.
- Review localStorage/sessionStorage contents.
- Protect routes while preserving backend authorization.
- Prevent accidental sensitive data in console logs.

CORS/security headers:

- Replace permissive production CORS with explicit configured origins.
- Do not use wildcard origins with credentialed requests.
- Review trusted hosts.
- Add appropriate security headers through the existing deployment architecture where applicable.

Rate limiting/abuse:

- Identify authentication, AI, booking, OTP-like, expensive search, and public endpoints that may require rate limiting.
- Add bounded protections without blocking legitimate workflows.
- Ensure rate limiting does not break idempotent booking recovery.

Audit:

- Record security-relevant authorization failures where appropriate.
- Do not log secrets or sensitive payloads.
- Preserve correlation IDs for investigation.

Testing:

- authentication bypass
- broken object-level authorization
- cross-tenant access
- privilege escalation
- forged identifiers
- prompt injection against capabilities
- AI unauthorized action attempt
- unsafe file path/name
- SQL injection inputs
- oversized pagination
- malformed datetime

- invalid state transition
- duplicate mutation
- secret leakage scan
- production frontend environment inspection

Run security-oriented tests and static searches across the repository. Fix confirmed vulnerabilities and document remaining accepted risks.

Prompt 39

Implement the CareFlow AI reliability and resilience engineering layer for database transactions, Redis failures, Celery failures, external EHR failures, notification failures, AI provider failures, network interruptions, duplicate messages, and service restarts. Inspect the existing reliability abstractions and avoid introducing multiple competing retry/idempotency systems.

The objective is graceful degradation and recoverability without corrupting business state.

Database reliability:

- Use explicit transaction boundaries.
- Keep database transactions short.
- Never hold a transaction open while waiting on slow external HTTP calls.
- Roll back failed business operations cleanly.
- Ensure partial writes cannot leave impossible appointment states.
- Use row locking/unique constraints where necessary for concurrency.
- Verify concurrent booking behavior under realistic load.

Booking transaction pattern:

- Validate authenticated/tenant context.
- Validate appointment resources.
- Validate current availability.
- Create/prepare internal operation state.
- Commit internal state required for external processing.
- Perform external integration outside the long database transaction.
- Persist external result.
- Verify external state.
- Synchronize final state.
- Trigger downstream workflows asynchronously.

Redis:

- Identify every Redis dependency.
- Separate durable business state from cache/context.
- If Redis is unavailable, critical persisted operations must not lose business truth.
- AI context may degrade gracefully to a persisted/reconstructed context path where supported.
- Avoid making Redis availability an unnecessary dependency for simple authenticated CRUD.
- Handle reconnect/backoff.
- Do not spin in infinite retry loops.

Celery:

- Ensure tasks are idempotent.
- Use bounded retries with exponential backoff/jitter where appropriate.
- Classify errors into retryable/permanent/unknown.
- Prevent duplicate task effects.

- Ensure task state can be recovered after worker restart.
- Ensure Beat schedules do not create duplicate logical work.
- Do not rely on in-memory worker state for durable business decisions.

External EHR:

- Define explicit timeout categories.
- Distinguish:

validation/authentication failure business rejection transient network failure timeout with unknown outcome malformed response external system unavailable external conflict

- Unknown outcome must query external state before retry.
- Safe retry must preserve logical operation identity.
- Never blindly repeat appointment creation after an unknown response.
- Verify external state before declaring confirmation.
- Preserve reconciliation when state diverges.

Notification providers:

- Retry transient delivery failures.
- Do not retry permanent invalid-recipient errors indefinitely.
- Deduplicate event-driven notifications.
- Keep notification failure separate from appointment failure.
- Do not roll back a confirmed appointment because email/SMS delivery failed.

AI providers:

- If an LLM provider fails, the underlying deterministic APIs and booking system must remain functional.
- Do not make AI a dependency for direct booking APIs.
- Handle provider timeout/rate-limit/unavailable errors safely.
- Use configured fallback providers only if explicitly supported.
- Do not silently change business behavior because a model failed.
- Preserve conversation state even if one model call fails.

Frontend/network:

- Handle temporary network failure without creating duplicate mutations.
- Show pending state where operation status is known.
- Refresh authoritative state after reconnect.
- Do not retry non-idempotent operations blindly.
- Preserve safe user context across route refresh.

Event/workflow reliability:

- Events must be durable enough for required downstream processing.
- Consumers must be idempotent.
- Duplicate event delivery must not duplicate business effects.
- If the project does not yet use an outbox pattern, assess whether one is necessary for reliable event publication; implement it only where justified by the current architecture.
- Never acknowledge an event before the required durable processing point.

Failure injection: Extend the Mock EHR and test infrastructure with deterministic scenarios:

- immediate success
- validation failure
- authentication failure
- timeout before creation
- timeout after external creation
- malformed response
- external conflict
- external appointment found during recovery
- external appointment absent during recovery
- divergent external state
- transient unavailable
- repeated failure

Add failure scenarios for:

- Redis unavailable
- Celery worker stopped
- notification provider unavailable
- AI provider unavailable
- PostgreSQL connection interruption where testable

Testing:

- restart recovery
- duplicate task
- duplicate event
- duplicate HTTP mutation
- EHR unknown outcome
- reconciliation
- booking concurrency
- Redis degradation
- worker restart
- provider timeout
- notification failure
- AI failure
- network reconnect

Document each retry boundary and explicitly state what is and is not safe to retry.

Prompt 40

Implement the CareFlow AI deployment, configuration, migration, backup/recovery, and environment-readiness layer for local development, staging, and production. Inspect the existing Dockerfiles, Docker Compose files, environment configuration, Alembic migrations, Railway/Vercel or other deployment configuration, Celery startup, health checks, and documentation before making changes.

The objective is to make deployment repeatable and prevent environment-specific behavior from changing business correctness.

Configuration:

- Centralize backend configuration through the existing settings system.
- Validate required production environment variables at startup.
- Provide safe defaults only for non-sensitive development settings.
- Never hard-code secrets.
- Maintain a complete `.env.example` with names, descriptions, and safe placeholder values.
- Clearly distinguish development, test, staging, and production configuration.
- Never expose backend secrets through frontend build-time variables.

Database:

- PostgreSQL is the durable source of truth.
- Validate `DATABASE_URL`/equivalent configuration.
- Use Alembic for schema migrations.
- Migrations must be forward-only and reproducible.
- Do not run destructive schema resets during application startup.
- Do not silently recreate production tables.
- Add migration checks to deployment verification.
- Verify indexes and constraints required for scheduling/idempotency/tenant isolation.

Migration safety:

- Review all existing migrations for ordering/dependency issues.
- Test upgrade from a clean database.
- Test upgrade from the current representative schema.
- Where practical, test downgrade only if the project explicitly supports it; do not pretend every production migration is safely reversible.
- Ensure application startup does not depend on a migration race between multiple replicas.
- Use a controlled migration step/job for production deployment.

Docker:

- Ensure backend image is reproducible.
- Ensure frontend production images/builds are reproducible where applicable.
- Ensure worker and Beat use the correct backend configuration.
- Ensure services communicate through service names inside Docker.
- Do not use localhost for inter-container communication.

- Add health checks appropriate to each service.
- Avoid running the application as root where practical.
- Keep images minimal without sacrificing debuggability.
- Do not bake secrets into images or layers.

Docker Compose development:

- PostgreSQL
- Redis
- backend
- Celery worker
- Celery Beat
- frontend applications as appropriate for the existing development setup

Use persistent volumes for PostgreSQL development data. Do not make Redis persistence a substitute for PostgreSQL business data.

Startup:

- Backend startup should be deterministic.
- Worker should load tasks reliably.
- Beat should not create duplicate schedules when restarted.
- Health/readiness endpoints should report meaningful status.
- Startup failures should be visible and actionable.

Deployment:

- Review the existing deployment target configuration rather than assuming a provider.
- Preserve current working deployment choices.
- Ensure backend CORS contains only intended frontend origins.
- Ensure frontend API base URLs are environment-specific.
- Ensure production frontend never points to localhost.
- Ensure worker/Beat deployment is accounted for where asynchronous workflows are required.
- Ensure persistent PostgreSQL is external/managed appropriately in production.
- Ensure Redis configuration is appropriate for production.

Secrets:

- Database credentials
- JWT signing keys
- EHR credentials
- notification provider credentials
- AI provider keys
- storage credentials

must come from the deployment secret mechanism.

Never:

- `commit .env``
- print secrets during startup
- return secrets from `/health``

- put secret values in frontend bundles
- log authorization headers

Backup/recovery:

- Document PostgreSQL backup expectations.
- Document restore procedure.
- Identify which data must be backed up:

users/auth metadata hospitals doctors calendars/availability appointments questionnaire definitions/responses
conversations integration mappings/operations reconciliation records audit records workflow state where durable

- Redis should not be treated as the only copy of critical data.
- Document recovery expectations after PostgreSQL loss.
- Document how pending EHR operations are reconciled after application recovery.

Observability/deployment:

- Preserve correlation IDs.
- Log structured operational events without sensitive payloads.
- Expose safe health/readiness information.
- Ensure worker failures are visible.
- Ensure scheduled task failures are visible.
- Ensure integration failures are queryable.

Deployment verification checklist:

- 1 clean database migration
- 2 existing database migration
- 3 backend startup
- 4 worker startup
- 5 Beat startup
- 6 frontend production build
- 7 CORS validation
- 8 authentication
- 9 tenant isolation
- 10 hospital lifecycle
- 11 doctor lifecycle
- 12 availability
- 13 booking
- 14 EHR verification
- 15 unknown-outcome recovery
- 16 questionnaire
- 17 notification
- 18 AI conversation
- 19 AI capability authorization
- 20 health/readiness
- 21 backup/restore documentation

Perform the final deployment smoke test using the real configured architecture, not a simplified fake path. Record exact commands and environment assumptions in the project documentation.

Do not claim deployment readiness if a required worker, database, Redis, migration, secret, integration, or frontend configuration is missing. Clearly identify blockers and provide the exact configuration or implementation required to resolve them.

Prompt 41

Implement the CareFlow AI comprehensive end-to-end automated testing strategy across backend, frontend, integrations, workflows, AI orchestration, scheduling, and deployment. Inspect all existing tests and test infrastructure before creating new suites. Reuse existing fixtures, factories, database setup, mock providers, and utilities instead of creating parallel test frameworks.

The goal is to verify complete business journeys rather than only isolated functions.

Establish a layered test structure:

- unit tests for domain rules
- service tests for application/domain orchestration
- API integration tests
- database/migration tests
- integration connector tests
- workflow/Celery tests
- AI capability/orchestrator tests
- frontend component tests
- frontend route/page tests
- browser end-to-end tests
- concurrency/reliability tests
- deployment smoke tests

Create reusable test fixtures for:

- platform admin
- operations admin
- hospital admin
- doctor
- patient
- second hospital
- departments
- specialties
- services
- appointment types
- calendars
- availability rules
- blocked periods
- appointments
- questionnaire definitions
- questionnaire responses
- conversations
- AI capabilities
- mock EHR records

- notification provider
- Redis/Celery test infrastructure

Use deterministic test data. Do not depend on real external AI, EHR, SMS, email, or payment providers for automated tests.

Authentication/authorization coverage:

- valid login
- invalid login
- expired token
- inactive user
- role mismatch
- missing authorization
- patient ownership
- doctor ownership
- hospital admin scope
- platform admin scope
- operations scope
- cross-hospital access
- forged resource IDs
- forged hospital IDs
- privilege escalation attempts

Hospital lifecycle coverage:

- draft
- submit
- review
- approve
- reject
- correction/resubmission
- suspend/reinstate where implemented
- doctor activation blocked before hospital activation
- booking blocked when hospital is not operational

Doctor lifecycle coverage:

- invited
- active
- inactive
- suspended/reinstated where implemented
- active doctor under inactive hospital
- doctor assignment to wrong hospital
- protected profile fields

Scheduling coverage:

- normal working hours
- outside working hours

- blocked period
- leave
- inactive calendar
- inactive doctor
- inactive hospital
- appointment type incompatibility
- consultation mode incompatibility
- duration boundaries
- exact end-time boundary
- overlapping appointment
- adjacent appointment
- timezone/DST behavior
- date range limits
- concurrent slot requests

Appointment coverage:

- request
- pending
- confirmed
- rescheduled
- completed
- no-show
- cancelled
- failed
- synchronization pending
- reconciliation required
- invalid state transition
- cancellation after completion where prohibited
- rescheduling after cancellation where prohibited
- duplicate booking request
- idempotent retry

EHR coverage:

- successful create
- successful verify
- successful synchronization
- validation failure
- authorization failure
- timeout before external creation
- timeout after external creation
- external appointment found during recovery
- external appointment absent
- external mismatch

- malformed response
- transient external failure
- permanent external failure

Verification must prove that the system does not report confirmation before external state is verified.

Recovery coverage:

- unknown outcome
- query-before-retry
- safe retry
- duplicate prevention
- reconciliation required
- operator retry
- operator resolution
- retry limit
- exponential backoff behavior where applicable

Questionnaire coverage:

- assignment
- patient authorization
- draft/save
- resume
- validation
- final submission
- duplicate submission
- immutable/versioned response
- doctor readiness visibility
- cross-patient denial
- cancelled appointment behavior

Notification coverage:

- event-triggered notification
- duplicate event
- provider success
- transient failure
- permanent failure
- retry
- cancellation of stale reminder
- rescheduling reminder update
- patient preference
- hospital configuration
- notification isolation

AI coverage:

- conversation creation
- conversation persistence

- conversation authorization
- context reconstruction
- capability authorization
- hospital-first discovery
- doctor discovery
- explore-more behavior
- appointment type selection
- real availability retrieval
- explicit booking confirmation
- successful booking
- unknown EHR outcome
- recovery
- duplicate request
- invalid doctor
- invalid appointment type
- stale slot
- unsupported clinical request
- prompt injection attempt
- AI provider failure
- AI provider timeout
- fallback behavior where configured

Prove that AI:

- cannot access SQL directly
- cannot access EHR vendors directly
- cannot bypass scheduling
- cannot bypass appointment state transitions
- cannot invent availability
- cannot claim confirmation without authoritative state

Frontend E2E journeys:

- 1 Patient registers/logs in.
- 2 Patient discovers an eligible hospital.
- 3 Patient discovers a doctor.
- 4 Patient selects appointment type.
- 5 Patient sees real availability.
- 6 Patient selects a slot.
- 7 Patient confirms booking.
- 8 Backend creates/coordinates appointment.
- 9 EHR state is verified.
- 10 Patient sees the actual resulting state.
- 11 Questionnaire becomes available.
- 12 Patient completes questionnaire.

- 13 Doctor sees authorized readiness information.
- 14 Hospital Admin sees the operational appointment.
- 15 Notification is generated.
- 16 Appointment can be rescheduled/cancelled through authorized flow.

Add failure-path browser tests:

- slot becomes unavailable
- booking conflict
- backend timeout
- EHR unknown outcome
- reconciliation required
- authentication expiry
- network interruption
- duplicate click
- page refresh during pending booking

Concurrency tests:

- two patients attempt same slot
- same patient submits same booking twice
- same idempotency key repeated
- different idempotency keys for same slot
- simultaneous cancellation/reschedule where applicable

Ensure exactly one valid appointment can occupy a slot.

Add test isolation:

- Each test cleans its state.
- Tests must not rely on execution order.
- Parallel tests must not corrupt shared state.
- External provider mocks must reset between tests.

Add coverage reporting and identify critical-path coverage separately from total line coverage. Do not optimize for an arbitrary percentage at the expense of business-critical behavior.

Run the full suite and fix real defects discovered. Do not weaken assertions merely to make tests pass. Document:

- test commands
- required services
- test environment variables
- known flaky tests
- unsupported external integrations
- critical workflow coverage.

Prompt 42

Implement the CareFlow AI performance and scalability hardening for the core scheduling, appointment, discovery, AI capability, dashboard, and operational APIs. Inspect actual query patterns, database indexes, request paths, Celery tasks, Redis usage, and frontend data fetching before optimizing. Do not introduce infrastructure complexity without evidence that it is needed.

The goal is predictable performance under realistic concurrent usage while preserving correctness, authorization, and tenant isolation.

Identify critical request paths:

- authentication
- hospital discovery
- doctor discovery
- doctor profile
- availability calculation
- booking
- appointment list/detail
- questionnaire retrieval/submission
- notifications
- conversation history
- AI capability execution
- hospital dashboard
- doctor dashboard
- platform/operations dashboards
- reconciliation operations

For each path determine:

- database queries
- external HTTP calls
- Redis operations
- Celery dependencies
- expected response size
- authorization checks
- transaction duration
- potential N+1 behavior

Database optimization:

- Add indexes based on real filtering/join/order patterns.
- Review indexes for hospital_id, doctor_id, patient_id, appointment state, appointment time ranges, lifecycle states, operation IDs, idempotency keys, external IDs, correlation IDs where appropriate.
- Ensure uniqueness constraints support correctness.
- Avoid unnecessary indexes that make writes expensive.
- Use composite indexes where query patterns justify them.

- Use EXPLAIN/EXPLAIN ANALYZE on representative queries.
- Fix sequential scans on large operational tables when they are caused by missing/selectivity-poor indexes.

Scheduling performance:

- Do not generate huge numbers of slots unnecessarily.
- Bound availability query windows.
- Filter inactive doctors/calendars before expensive slot calculations.
- Efficiently subtract working hours, blocks, leave, and appointments.
- Avoid loading unrelated appointments into memory.
- Ensure overlap queries use indexed time ranges where supported.
- Preserve exact boundary correctness while optimizing.

Booking concurrency:

- Measure and test contention around popular slots.
- Use database constraints/locking appropriate to the current architecture.
- Keep lock scope minimal.
- Do not trade correctness for throughput.
- Verify that high concurrency still results in one valid booking.

Discovery:

- Add pagination.
- Add server-side filtering.
- Avoid loading all hospitals/doctors/services.
- Use selective response fields.
- Consider safe search indexes only where needed.

Dashboards:

- Avoid N+1 queries.
- Use aggregate queries for counts.
- Paginate detailed lists.
- Avoid returning large appointment collections merely to calculate totals.
- Cache only safe short-lived aggregate data where appropriate.
- Ensure cached results remain tenant-scoped.

AI:

- Limit context size to what the current task requires.
- Do not send entire conversation history when unnecessary.
- Use structured context rather than repeated raw data.
- Cache safe reference data only when appropriate.
- Avoid duplicate capability calls within a single turn.
- Bound model/tool loops.
- Prevent infinite agent loops.
- Set explicit timeouts.

External integrations:

- Configure connect/read timeouts.

- Avoid holding database transactions during external calls.
- Use bounded retries.
- Use connection pooling where appropriate.
- Prevent retry storms during external outages.

Redis:

- Set appropriate TTLs.
- Prevent unbounded key growth.
- Namespace keys by environment and resource scope.
- Never use Redis cache keys that can leak data across hospitals/users.
- Avoid using Redis as the durable source of truth.

Celery:

- Inspect task queue behavior.
- Avoid long-running tasks blocking short critical tasks where queues can be separated.
- Ensure task retries are bounded.
- Prevent duplicate periodic jobs.
- Avoid fetching massive datasets in one task.
- Use chunking/batching where justified.

Frontend performance:

- Avoid duplicate API calls.
- Avoid unnecessary full-page refetches.
- Use pagination for large lists.
- Debounce search where appropriate.
- Do not debounce booking mutations.
- Lazy-load large admin surfaces where useful.
- Avoid shipping server-only libraries or secrets to frontend bundles.
- Measure bundle size and remove obvious unused dependencies where safe.

Load testing: Create representative scenarios for:

- hospital/doctor discovery
- availability
- concurrent booking
- appointment list
- AI capability request
- dashboard aggregation

Define realistic concurrency and data volumes in the test documentation. Measure:

- latency percentiles
- error rate
- database utilization
- connection pool saturation
- Redis behavior
- Celery queue depth

- external integration latency

Do not invent performance numbers. Record measured results from the actual environment.

Resilience under load:

- Confirm rate limits where necessary.
- Confirm bounded queue growth.
- Confirm booking correctness during contention.
- Confirm tenant isolation under concurrent requests.
- Confirm external failure does not cause retry storms.

Run before/after measurements for meaningful optimizations and retain only changes that improve or preserve correctness and measured behavior.

Prompt 43

Implement the CareFlow AI accessibility, responsive UX, error-state, and usability hardening across all five frontend applications while preserving the existing design language and business architecture. Inspect current UI components and design-system primitives before creating new ones.

The objective is to make the product usable across realistic desktop/tablet/mobile contexts and accessible without changing backend business rules.

Apply consistent frontend behavior to:

- Patient
- Doctor
- Hospital Admin
- Operations Admin
- Platform Admin

Accessibility:

- Use semantic HTML.
- Ensure every interactive control has an accessible name.
- Associate labels with inputs.
- Ensure keyboard navigation works.
- Ensure visible focus states.
- Use appropriate heading hierarchy.
- Use buttons for actions and links for navigation.
- Use dialogs with correct focus management.
- Use accessible table semantics.
- Provide accessible status/live regions for asynchronous state changes where appropriate.
- Do not rely on color alone to communicate appointment or lifecycle status.
- Provide text/status labels alongside visual indicators.
- Ensure form validation errors are associated with their fields.
- Ensure disabled/loading states are understandable.
- Support screen-reader-friendly booking state updates.

Keyboard workflows:

- Patient can search, select hospital, select doctor, choose appointment type, choose date/time, review booking, and confirm without a mouse.
- Doctor can navigate appointments/calendar.
- Hospital Admin can manage configuration and doctors.
- Operations/Admin can review/retry/resolve operational records.
- Platform Admin can review hospital applications.

Responsive behavior:

- Patient booking must remain usable on small screens.
- Calendar/time-slot selection must adapt without hiding important information.

- Admin tables should provide a usable mobile/tablet representation rather than overflowing uncontrollably.
- Chat/AI surfaces should remain usable on mobile.
- Dialogs should fit small screens.
- Navigation should collapse appropriately while preserving access to all routes.

Loading states:

- Use skeletons/placeholders for content-heavy pages where appropriate.
- Avoid flashing fake data.
- Disable duplicate actions while mutations are in flight.
- Preserve user-entered form state during recoverable errors.

Empty states:

- No hospitals found.
- No doctors found.
- No availability.
- No appointments.
- No questionnaires.
- No notifications.
- No reconciliation items.
- No workflow failures.
- No audit records.
- No analytics data.

Each empty state should explain what happened and provide a useful next action when possible.

Error states: Distinguish:

- network failure
- authentication expiry
- permission denied
- not found
- validation error
- scheduling conflict
- external synchronization pending
- reconciliation required
- server error

Do not show raw stack traces or internal exception messages.

Booking UX:

- Show selected hospital/doctor/appointment type clearly.
- Show date/time/timezone.
- Show appointment duration.
- Show consultation mode.
- Show final booking review before confirmation where required.
- Make pending/synchronization state visually distinct from confirmed.
- When a slot becomes unavailable, explain it and allow the patient to return to availability.

- Never show a success screen before the backend confirms the appropriate state.

AI UX:

- Clearly distinguish assistant messages from structured actions/cards.
- Make tool-driven actions understandable without exposing internal tool traces.
- Show when an operation is still processing.
- Provide recovery actions for failed/pending workflows.
- Preserve conversation context across navigation.
- Do not display internal prompts, secrets, stack traces, or raw capability payloads.

Forms:

- Use consistent validation.
- Preserve input on validation failure.
- Confirm destructive actions.
- Show success based on persisted backend state.
- Prevent accidental duplicate submissions.

Status language: Use consistent patient-safe terminology for appointment states. Internal operational terms such as reconciliation should be translated into understandable user-facing explanations where exposed.

Testing:

- keyboard navigation
- focus behavior
- form labels/errors
- screen-reader semantics where test tooling supports it
- responsive viewport tests
- loading/empty/error states
- booking conflict UX
- pending EHR state
- questionnaire submission
- admin tables
- AI chat
- all production builds

Do not rewrite the entire UI. Improve existing components systematically and keep visual consistency.

Prompt 44

Implement the CareFlow AI data lifecycle, privacy, retention, export, and deletion controls for user-owned and tenant-owned data using the existing domain models, authorization, audit, conversation, questionnaire, appointment, notification, and document structures. Inspect the current data model and requirements before implementing. Do not invent legal/compliance claims or promise retention periods that are not defined by the project.

The goal is controlled data ownership and lifecycle behavior.

Inventory persistent data categories:

- authentication/user records
- patient profile
- doctor profile
- hospital configuration
- appointments
- appointment history
- questionnaires
- questionnaire responses
- conversations
- AI context references
- capability executions
- notifications
- integration operations
- external mappings
- reconciliation records
- workflow executions
- audit events
- analytics/AI usage events
- uploaded documents/files where present

For each category document:

- owner
- tenant scope
- access roles
- whether it is mutable
- whether it is historical/audit data
- whether deletion is allowed
- whether anonymization is appropriate
- dependencies
- retention policy source, if one exists

Do not delete historical business records merely because a user-facing entity is removed.

User data access:

- Patient can access only their permitted profile, appointments, questionnaire status/responses, conversations, notifications, and other explicitly supported personal data.
- Doctor can access only their own profile and authorized operational records.
- Hospital Admin can access only hospital-scoped records allowed by role.
- Platform Admin and Operations/Admin must access only the operational data required by their responsibilities.

Implement safe profile updates:

- Explicit mutable fields.
- Protected fields cannot be mass-assigned.
- Email/account identity changes follow the existing authentication policy.
- Hospital ownership and role cannot be changed by normal user profile updates.

Account deactivation:

- Prevent login after deactivation.
- Preserve necessary historical references.
- Ensure active appointments and workflows have a defined behavior.
- Do not silently delete appointment history.
- Ensure scheduled notifications/workflows do not continue incorrectly for deactivated users.

Conversation lifecycle:

- Store only required durable conversation information.
- Ensure conversations remain user/tenant scoped.
- Provide deletion/archive behavior only if supported by the project requirements.
- Deleting a conversation must not delete authoritative appointment/business records.
- AI context cache should expire independently from durable conversation history.

Questionnaire lifecycle:

- Responses remain associated with the correct appointment/questionnaire version.
- Do not modify submitted responses as if they were drafts.
- If corrections are supported, use an explicit version/history mechanism.
- Deleting UI references must not corrupt appointment history.

Document/file lifecycle where applicable:

- Tenant/owner authorization.
- Safe deletion.
- No orphaned database records.
- No orphaned storage objects where cleanup is supported.
- No direct public storage URLs for private documents unless the architecture explicitly provides controlled access.

Export: If the product requirements support data export, implement an asynchronous export workflow rather than generating a huge synchronous HTTP response.

Export must:

- verify authorization
- define exact data scope
- avoid exposing other users/tenants
- record export request

- generate through controlled background processing
- provide a time-limited authorized retrieval mechanism
- avoid logging exported sensitive contents

If export is not currently required, create the domain boundary/documentation without pretending the feature exists.

Deletion: Do not implement irreversible deletion of critical healthcare/appointment/audit data merely for completeness. Where deletion is supported, explicitly classify:

- hard delete
- soft delete
- anonymization
- retention-only record

Ensure foreign-key relationships and audit integrity remain valid.

Privacy-safe logging:

- Search for sensitive payloads in logs.
- Remove raw questionnaire answers.
- Remove medical details where not necessary.
- Remove full conversation payloads from operational logs.
- Remove tokens/passwords/credentials.
- Minimize AI prompt/response logging.

Testing:

- access isolation
- profile update authorization
- deactivation behavior
- conversation isolation
- questionnaire privacy
- document privacy
- export scope
- deletion/anonymization behavior where implemented
- no sensitive payloads in audit/log records
- cross-tenant privacy

Document the actual implemented data lifecycle and clearly identify anything that remains intentionally outside the system.

Prompt 45

Implement the final CareFlow AI product integration, demo readiness, seeded realistic environment, operational walkthrough, and architecture validation layer. Inspect the entire repository and all work from the previous prompts before changing anything. This is an integration and verification task, not an opportunity for a broad rewrite.

Create a deterministic demo environment that represents the complete product journey without using fake behavior in production code.

Seed realistic but clearly synthetic data for:

- at least two hospitals
- multiple departments
- multiple specialties
- multiple services
- multiple appointment types
- multiple doctors
- doctor calendars
- recurring working hours
- blocked periods/leave
- patients
- appointments in multiple lifecycle states
- questionnaire definitions
- sample questionnaire responses where appropriate
- notifications
- conversations
- mock EHR patients/providers/facilities/appointments
- integration operations
- at least one reconciliation/failure scenario
- operational audit records

The second hospital must be used to verify tenant isolation.

Create deterministic demo accounts for each supported role through the existing safe development mechanism:

- Patient
- Doctor
- Hospital Admin
- Operations Admin where implemented
- Platform Admin

Never commit real passwords, secrets, API keys, or production credentials.

Build a complete demo walkthrough:

PATIENT JOURNEY

- 1 Login/register.
- 2 Find care.

- 3 Select eligible hospital.
- 4 Discover doctor.
- 5 Inspect doctor details.
- 6 Select appointment type.
- 7 Select date.
- 8 See real available slots.
- 9 Select slot.
- 10 Review booking.
- 11 Confirm booking.
- 12 Observe appointment pending/verification state if applicable.
- 13 Observe final verified state.
- 14 Open appointment details.
- 15 Complete questionnaire.
- 16 View notifications.
- 17 Reschedule.
- 18 Cancel where appropriate.

DOCTOR JOURNEY

- 1 Login.
- 2 View dashboard.
- 3 View calendar.
- 4 View availability.
- 5 View upcoming appointments.
- 6 Open authorized appointment details.
- 7 View questionnaire readiness.
- 8 Review authorized questionnaire information if supported.
- 9 Complete/mark appointment.
- 10 Observe resulting notification/workflow behavior.

HOSPITAL ADMIN JOURNEY

- 1 Login.
- 2 View hospital dashboard.
- 3 Configure departments/specialties/services.
- 4 Configure appointment types.
- 5 Manage doctors.
- 6 Configure calendar/availability.
- 7 View operational appointments.
- 8 Configure communication.
- 9 View safe integration status.
- 10 Review operational/audit information.

PLATFORM ADMIN JOURNEY

- 1 Login.
- 2 Review hospital application.
- 3 Approve/reject/request correction as appropriate.
- 4 View platform-level status.
- 5 Verify hospital lifecycle audit events.

OPERATIONS JOURNEY

- 1 Login.
- 2 View failed integration operations.
- 3 Inspect unknown EHR outcome.
- 4 Query external state.
- 5 Recover safely.
- 6 Observe synchronization.
- 7 Resolve reconciliation where required.
- 8 Verify audit/correlation trace.

AI JOURNEY

- 1 Patient starts conversation.
- 2 AI resolves intent.
- 3 AI discovers hospital.
- 4 AI discovers doctor.
- 5 User explores more doctors.
- 6 User selects appointment type.
- 7 AI retrieves real availability.
- 8 User selects a slot.
- 9 AI requests explicit booking confirmation.
- 10 Booking capability executes.
- 11 EHR verification occurs.
- 12 AI reports actual appointment state.
- 13 Questionnaire workflow becomes available.
- 14 AI can report questionnaire status.
- 15 AI never fabricates availability or confirmation.

FAILURE DEMO

Create one deterministic demonstration of:

- external EHR timeout after external appointment creation
- internal state remains recoverable/pending
- external state query discovers appointment
- synchronization completes
- duplicate appointment is not created
- final patient-facing state becomes correct
- correlation/operation IDs trace the entire chain

Also demonstrate the divergent-state path in automated tests even if it is not part of the default happy-path demo.

Demo environment requirements:

- one documented startup command
- one documented seed command
- one documented test command
- one documented reset/cleanup command for development only
- no destructive reset command should run automatically
- environment variables documented
- service dependencies documented
- backend/worker/Beat/frontend startup documented

Create a product demo checklist that confirms:

- authentication works
- tenant isolation works
- hospital lifecycle works
- doctor lifecycle works
- availability is real
- booking is concurrency-safe
- EHR verification is explicit
- unknown outcome recovery works
- questionnaire works
- notifications work
- AI capabilities are authorized
- AI cannot bypass domain services
- audit/correlation tracing works
- dashboards show persisted data
- all five applications build
- deployment configuration is documented

Architecture validation: Trace at least one complete request through: frontend ? API ? authentication ? tenant context ? application service ? AI/capability where applicable ? scheduling ? appointment ? EHR connector ? verification ? synchronization ? event/workflow ? questionnaire/notification ? audit/analytics.

Verify that no layer bypasses the intended architecture.

Search the repository for:

- duplicate appointment implementations
- duplicate scheduling logic
- fake slot generators
- hard-coded demo responses in production paths
- direct EHR calls outside integration
- direct database calls from AI
- unscoped tenant queries
- generic status setters
- secrets

- stale environment variables
- obsolete AI provider assumptions
- unused feature flags
- dead routes
- dead frontend pages
- inconsistent API clients

Remove only confirmed dead/duplicate code. Do not delete uncertain code merely because it appears unused.

Produce final documentation:

- system architecture
- application boundaries
- role matrix
- booking lifecycle
- EHR integration/verification
- failure recovery
- questionnaire workflow
- notification workflow
- AI capability architecture
- data lifecycle
- deployment
- demo credentials for development only
- test commands
- troubleshooting guide

Run the final verification suite and record actual results:

- backend unit/integration tests
- frontend tests
- typecheck
- production builds
- migration validation
- Docker build/start
- health/readiness
- Celery worker/Beat
- concurrency booking tests
- EHR failure/recovery tests
- tenant isolation tests
- representative browser E2E journey

Do not claim that every requirement is implemented if evidence is missing. The final report must distinguish:

- verified
- implemented but not fully tested
- intentionally deferred
- known limitations

The final product should demonstrate one coherent CareFlow AI system rather than a collection of disconnected features.

Prompt 46

Implement the CareFlow AI search and discovery architecture as a unified backend capability used by Patient, Doctor, Hospital Admin, and AI workflows. Inspect the existing hospital, department, specialty, service, doctor, appointment type, and discovery APIs before changing anything. Do not create separate discovery logic for the UI and AI.

The objective is to make discovery authoritative, tenant-safe, searchable, paginated, deterministic, and reusable.

Define a discovery/application layer that can support:

- hospital discovery
- department discovery
- specialty discovery
- service discovery
- doctor discovery
- appointment type discovery
- doctor profile retrieval
- supported consultation modes
- availability eligibility information where appropriate

Hospital discovery:

- Return only hospitals eligible for patient-facing discovery.
- Respect hospital lifecycle state.
- Exclude draft/rejected/suspended hospitals unless the requesting role explicitly has permission to view them.
- Support supported filters such as location/service/specialty/department where the existing domain provides them.
- Never expose private hospital configuration.
- Use pagination.
- Return stable identifiers and patient-safe display fields.

Doctor discovery:

- Only return doctors eligible for the requested operation.
- Validate hospital scope.
- Respect doctor lifecycle.
- Filter by hospital, department, specialty, service, appointment type, consultation mode, language, or other existing supported fields.
- Do not return inactive doctors to patient booking flows.
- Do not expose internal/external provider identifiers unless explicitly patient-safe.
- Return enough information for a doctor card without leaking private administrative fields.

Service/appointment-type discovery:

- Only return active, hospital-approved configuration.
- Ensure appointment type belongs to the selected hospital.
- Ensure the doctor can actually perform the selected appointment type before presenting it as bookable.

- Do not allow the frontend or AI to invent appointment duration.

Search:

- Use server-side filtering and pagination.
- Define deterministic ordering.
- Support query text only where appropriate.
- Normalize safe searchable fields.
- Avoid expensive wildcard scans on large datasets without appropriate indexes.
- Add database indexes based on actual search patterns.
- Enforce maximum page sizes.

Discovery and availability must remain separate:

- Discovery identifies eligible resources.
- Scheduling determines real availability.
- Never return a doctor as “available” merely because the doctor is active.
- Never display generated fake time slots.

AI integration:

- Expose discovery through typed capabilities.
- AI receives structured results, not direct database records.
- AI must use the same discovery source as the frontend.
- “Explore more” must use deterministic pagination/cursor behavior.
- Avoid returning duplicate doctors across consecutive result sets.
- Preserve search criteria in AIContext.
- Validate any selected resource against current backend state before proceeding.

Selection validation: When a patient/AI selects:

- hospital
- doctor
- service
- appointment type

validate that the resource:

- exists
- is active
- belongs to the correct hospital
- is compatible with the requested workflow
- is accessible to the authenticated actor

If a selected resource becomes inactive between discovery and selection, return a structured conflict and require rediscovery.

Doctor profile:

- Return patient-safe profile information.
- Return doctor's supported services/appointment types only when currently active.
- Do not expose internal scheduling implementation details.

Hospital profile:

- Return patient-safe information only.
- Separate public discovery information from private admin configuration.

Caching:

- Cache only safe reference/discovery data.
- Cache keys must include appropriate scope and filter parameters.
- Never allow one hospital's cached result to be returned to another tenant.
- Apply short TTLs where data changes frequently.
- Invalidate relevant cache after hospital/doctor/service configuration changes.

Testing:

- patient sees only discoverable hospitals
- inactive hospital excluded
- suspended hospital excluded
- patient sees only active doctors
- cross-hospital doctor lookup denied
- inactive service excluded
- appointment type compatibility
- deterministic pagination
- no duplicate "explore more" results
- stale selection conflict
- tenant-safe cache
- AI and frontend return consistent discovery data
- large result pagination
- authorization for admin-only discovery states

Document the discovery contract and ensure all consuming applications use the same backend capability rather than implementing independent filtering rules.

Prompt 47

Implement the CareFlow AI appointment rescheduling and cancellation workflows as complete, concurrency-safe, EHR-aware business operations. Inspect the existing appointment state machine, scheduling engine, integration connector, verification/synchronization services, notification workflows, and frontend flows before making changes.

Do not create independent reschedule/cancel logic in the frontend, AI layer, or EHR connector.

Cancellation:

- Define which appointment states can be cancelled.
- Enforce cancellation policy through explicit domain/application methods.
- Validate actor authorization.
- Validate patient/doctor/hospital ownership.
- Reject invalid or terminal-state cancellation.
- Persist the cancellation request/state correctly.
- Coordinate external cancellation when an external appointment exists.
- Verify the external result before finalizing an externally dependent state.
- Keep internal and external states traceable.

Rescheduling:

- Validate the current appointment state.
- Validate actor authorization.
- Require a new valid date/time.
- Recalculate availability using the central scheduling engine.
- Never trust a frontend-provided slot as available.
- Revalidate immediately before committing the reschedule.
- Prevent collision with another appointment.
- Respect working hours, blocked periods, leave, calendar state, appointment type duration, consultation mode, and external constraints.
- Preserve the original appointment history.
- Record the old and new schedule safely.

EHR rescheduling: Use the existing connector abstraction:

- update/reschedule externally where supported
- verify external result
- synchronize internal state
- handle unsupported vendor behavior through the integration service
- never expose vendor-specific logic to frontend/AI

If the external system performs rescheduling as cancel + create:

- keep that behavior inside the integration layer
- use the same logical operation identity
- prevent duplicate external appointments
- recover safely if the external outcome becomes unknown

Unknown outcome:

- Do not blindly retry external cancellation/rescheduling.
- Query external state first.
- Compare expected appointment identity/time/status.
- Synchronize if the desired state already exists.
- Retry only when safe.
- Mark reconciliation required for divergent states.

Concurrency:

- Two reschedule operations targeting the same new slot must not both succeed.
- A cancellation racing with rescheduling must resolve through explicit state rules.
- A booking racing with rescheduling must preserve scheduling correctness.
- Use transaction/locking/constraints appropriate to the existing implementation.

Appointment history: Record:

- previous state
- new state
- previous start/end
- new start/end for reschedule
- actor
- timestamp
- correlation ID
- operation ID
- safe reason where provided

Do not overwrite historical appointment events.

Notifications:

- Cancellation must invalidate obsolete reminders.
- Rescheduling must update future reminders.
- Notification delivery failure must not reverse a valid appointment state.
- Duplicate events must not generate duplicate logical notifications.

Questionnaires:

- Determine what happens to an already assigned questionnaire after rescheduling.
- Preserve questionnaire history.
- Do not silently attach an old response to an unrelated appointment.
- If cancellation makes a questionnaire irrelevant, transition it through an explicit supported workflow state.

Frontend: Patient:

- appointment detail ? cancel/reschedule
- clear current state
- available dates/slots from real scheduling
- review new time
- confirmation/pending state
- final verified state

Doctor:

- authorized reschedule/cancel actions
- clear state transitions
- no arbitrary status mutation

Hospital Admin:

- authorized operational rescheduling/cancellation
- same backend service and validation

AI:

- use the same cancellation/reschedule capabilities
- require sufficient information
- present the resulting state from the backend
- never claim completion before the operation reaches the appropriate state

Testing:

- cancellation for every relevant state
- invalid cancellation
- authorized/unauthorized cancellation
- successful reschedule
- stale slot
- conflicting slot
- concurrent reschedule
- EHR success
- EHR unknown outcome
- EHR mismatch
- recovery
- notification update
- questionnaire behavior
- idempotent repeated cancellation
- idempotent repeated reschedule request
- cross-tenant denial

Document the state transitions and recovery rules.

Prompt 48

Implement the CareFlow AI centralized appointment timeline and operational history model so that patients, doctors, hospital administrators, operations users, and platform administrators can understand the correct lifecycle of an appointment without exposing unauthorized or sensitive information.

Inspect the current appointment, history, audit, integration operation, verification, synchronization, workflow, and notification models before implementation. Reuse existing records where possible instead of creating redundant histories.

Create one coherent timeline abstraction that can represent meaningful appointment events such as:

- appointment requested
- appointment created
- availability reserved/validated where applicable
- EHR create initiated
- EHR create completed
- external verification started/completed
- appointment confirmed
- synchronization completed
- questionnaire assigned
- questionnaire submitted
- appointment reschedule requested/completed
- appointment cancellation requested/completed
- reminder scheduled/sent
- appointment completed
- no-show
- failure
- synchronization pending
- reconciliation required
- reconciliation resolved

Do not expose every internal implementation event to every user.

Separate:

- patient-safe timeline
- doctor operational timeline
- hospital operational timeline
- operations/integration timeline
- platform/audit timeline

Patient timeline: Show understandable events such as:

- booking requested
- booking confirmed
- appointment rescheduled

- questionnaire available
- questionnaire completed
- appointment cancelled
- appointment completed

Do not expose:

- raw EHR payloads
- internal stack traces
- secret/provider details
- internal retry mechanics
- another actor's private information
- sensitive operational metadata not required by the patient

Doctor timeline: Show authorized appointment lifecycle and operational readiness information.

Hospital Admin: Show hospital-scoped appointment lifecycle and operational events.

Operations: Show integration/verification/reconciliation information needed for recovery.

Platform Admin: Show platform governance/audit information according to authorization.

Timeline event model should contain safe structured metadata:

- event ID
- appointment ID
- event type
- actor type/actor ID where appropriate
- event timestamp
- effective time if different
- correlation ID
- operation ID
- safe summary
- source category
- resulting appointment state where relevant

Do not store arbitrary JSON payloads without schema or size controls.

Ordering:

- Define deterministic ordering when timestamps are equal.
- Preserve event chronology.
- Distinguish event creation time from effective business time where required.

Idempotency:

- Duplicate workflow/event delivery must not create duplicate timeline entries for the same logical event.
- Use stable event identity or equivalent deduplication.

Integration with existing audit:

- Do not copy all audit events into appointment timeline.
- Create a mapping/filter layer for user-facing history.
- Keep immutable audit records separate from presentation-oriented timeline data.

API: Provide appointment timeline retrieval through a dedicated authorized endpoint or appropriate appointment detail response.

- Paginate if history can grow large.
- Enforce actor/resource scope.
- Return typed event models.
- Avoid returning raw internal event payloads.

Frontend: Patient appointment detail should show a simple chronological timeline. Doctor/hospital/operations surfaces can show progressively more operational detail according to authorization. Use accessible status indicators and timestamps.

AI:

- AI can query a controlled appointment-history capability.
- AI should summarize only the events available to the authenticated actor.
- Do not allow AI to infer hidden events.
- If the current state is uncertain, report the authoritative pending/reconciliation state.

Testing:

- chronological ordering
- duplicate event prevention
- patient privacy
- doctor scope
- hospital scope
- operations scope
- cross-tenant denial
- EHR event visibility rules
- reconciliation timeline
- cancellation/reschedule history
- questionnaire events
- pagination
- AI timeline capability

Document which events are durable business history, audit-only, operational-only, and patient-visible.

Prompt 49

Implement the CareFlow AI document and knowledge foundation for administrative/access workflows using the existing project architecture, storage boundaries, AI context, capability layer, tenant isolation, and security controls. Only implement document functionality that is actually supported by the existing product requirements; do not turn the platform into a general medical document or clinical decision system.

Inspect existing document/file models, upload endpoints, storage configuration, frontend document surfaces, AI/RAG code, and security controls before implementation.

Supported document categories may include, where already required:

- hospital administrative documents
- service/appointment information
- hospital policies
- doctor/service information
- patient-provided administrative documents
- workflow/reference material

Do not assume that every uploaded document is safe to expose to every AI workflow.

Document model:

- document ID
- owner/resource scope
- hospital ID where applicable
- patient ID where applicable
- document category
- original filename
- safe storage key
- content type
- size
- checksum where useful
- processing status
- extraction status
- indexing status
- version
- created/updated timestamps
- deletion/archive state where supported

Storage:

- Never use the original filename as the storage path.
- Prevent path traversal.
- Generate server-side storage identifiers.
- Keep private documents outside public static directories.
- Enforce authorization before download/read.

- Ensure tenant scope is included in storage lookup.
- Do not expose storage credentials to frontend.

Upload validation:

- maximum file size
- allowed content types
- safe filename handling
- extension/content validation where supported
- reject unsupported executable content
- validate empty/corrupt files
- prevent duplicate uploads where useful through checksum/versioning

Processing: Use the existing background infrastructure for expensive work:

- 1 upload persisted
- 2 processing status set
- 3 background extraction
- 4 text extraction/normalization
- 5 optional OCR where already supported
- 6 chunking where required
- 7 embedding/indexing where required
- 8 processing complete/failed

Do not block a normal HTTP request on expensive document processing.

Processing states should be explicit:

- uploaded
- processing
- extracted
- indexed
- ready
- failed
- archived/deleted where supported

Failure recovery:

- retry transient processing failures
- do not endlessly retry corrupted/unsupported files
- preserve error category
- allow authorized retry where appropriate
- ensure duplicate task execution does not duplicate chunks/embeddings

Tenant isolation:

- Patient documents must never appear in another patient's retrieval context.
- Hospital documents must remain hospital-scoped.
- Doctor/admin retrieval must respect role and hospital scope.
- AI retrieval must enforce the same authorization as normal API access.

AI/RAG boundary:

- Retrieved document text is untrusted content.
- Document content must not override system/application authorization.
- Do not allow embedded instructions in documents to execute capabilities.
- Retrieval results should carry source/document identifiers.
- AI responses must distinguish retrieved information from generated interpretation.
- Do not use administrative documents to generate diagnosis/prescribing/clinical decisions.

Search/retrieval:

- Filter by authorized tenant/resource scope before semantic retrieval where possible.
- Apply authorization after retrieval as defense in depth.
- Avoid retrieving documents outside the current user's scope.
- Preserve source references.
- Limit context size.
- Avoid duplicate chunks.

Frontend:

- Document list
- upload
- processing status
- view/download where authorized
- retry failed processing where authorized
- archive/delete where supported
- search where supported

Do not expose raw storage paths.

Testing:

- upload validation
- path traversal
- unauthorized download
- cross-tenant retrieval
- processing failure
- retry
- duplicate processing
- document versioning
- AI retrieval authorization
- prompt injection through document content
- source/reference preservation
- private document protection

Document exactly which document types are supported and which AI workflows may consume them.

Prompt 50

Implement the CareFlow AI final architecture-governance and maintainability pass across the complete repository. Inspect the implementation produced by Prompt1–Prompt49 and compare the actual code structure against the intended PRD and architecture. Do not introduce new product features unless a concrete architecture gap blocks an existing requirement.

The objective is to leave the codebase understandable, modular, testable, and safe for continued development.

Perform a repository-wide inventory:

- backend modules
- frontend applications
- shared frontend packages
- database models
- migrations
- domain services
- scheduling
- AI/context
- MCP/capabilities
- integration/EHR
- verification/synchronization
- workflows/Celery
- notifications
- questionnaires
- documents
- audit/observability
- analytics
- tests
- deployment/infra
- documentation

For every major module verify:

- clear ownership
- single business responsibility
- explicit public interfaces
- no circular dependency
- no duplicate implementation
- no hidden database access
- no vendor-specific leakage
- test coverage appropriate to criticality

Architecture boundaries must remain:

Frontend ? API/Application ? Authentication/Tenant Context ? Domain/Application Services ?
Scheduling/Appointment ? Integration/Verification/Synchronization ? Workflow/Event System ?
Notifications/Questionnaires/Analytics/Audit

AI must remain:

AI UI/Input ? AI Orchestrator ? AIContext ? Typed Capabilities/MCP ? Authorized Application/Domain Services

AI must never become:

AI ? SQL ? EHR vendor ? arbitrary HTTP ? scheduling bypass ? appointment-state bypass

Review dependency direction:

- domain should not depend on frontend
- scheduling should not depend on AI
- appointment service should not depend on React/UI
- EHR connector should not contain patient-facing UI logic
- AI should depend on capabilities/application interfaces rather than vendor implementations
- notification providers should remain replaceable
- Celery tasks should call application services rather than duplicate business rules

Identify duplicated logic:

- appointment status transitions
- availability calculations
- authorization
- tenant filtering
- discovery filtering
- notification scheduling
- EHR error classification
- idempotency
- retry logic
- date/time conversion

Centralize only where duplication is confirmed. Avoid creating overly generic “utils” that hide business logic.

Naming consistency:

- consistent IDs
- consistent timestamps
- consistent status naming
- consistent operation/correlation terminology
- consistent hospital/doctor/patient ownership terminology
- consistent frontend route naming
- consistent API resource naming

Error model: Ensure application errors are:

- typed
- meaningful
- safe to expose
- mapped consistently at API boundaries

- distinguish authorization, validation, conflict, external failure, pending, and internal errors

Configuration:

- remove unused environment variables
- document required variables
- remove hard-coded environment-specific URLs
- ensure frontend/backend configuration is explicit
- ensure secrets are never committed

Database:

- inspect indexes
- inspect constraints
- inspect foreign keys
- inspect unique/idempotency constraints
- inspect migration consistency
- inspect tenant-scoping fields
- inspect historical/audit retention behavior

Concurrency: Review:

- booking
- rescheduling
- cancellation
- EHR create
- EHR recovery
- questionnaire submission
- notification delivery
- workflow execution

Ensure each critical mutation has an explicit concurrency/idempotency strategy.

Observability: Verify that correlation IDs can trace: request ? capability ? operation ? appointment ? integration ? verification ? synchronization ? workflow ? notification

Ensure logs are structured and sensitive data is excluded.

Testing architecture:

- Keep unit tests close to domain logic.
- Keep integration tests for boundaries.
- Keep E2E tests for complete journeys.
- Avoid tests that mock the entire system and therefore prove nothing.
- Avoid tests that depend on real external providers.
- Keep deterministic Mock EHR scenarios.

Frontend maintainability:

- remove duplicated API clients
- remove duplicated auth handling
- remove dead routes/components
- centralize shared UI primitives

- centralize shared types where appropriate
- keep business rules out of components
- avoid deeply nested prop drilling where a clear existing state pattern is available
- avoid global state for data that belongs in server state

AI maintainability:

- keep prompts/configuration versioned where appropriate
- keep capability schemas typed
- keep context structured
- keep tool results separate from natural-language output
- prevent prompt strings from containing business logic that should live in services
- keep model/provider configuration externalized
- ensure provider failure cannot corrupt domain state

Documentation: Update:

- README
- architecture documentation
- API documentation
- local development guide
- deployment guide
- environment configuration guide
- role/permission matrix
- booking workflow
- EHR verification/recovery
- AI capability architecture
- questionnaire workflow
- notification workflow
- document workflow
- observability guide
- testing guide
- troubleshooting guide

Documentation must describe the actual implementation, not an aspirational architecture.

Final code-quality checks:

- lint
- typecheck
- formatting
- unused imports
- unreachable code
- stale TODOs
- dead feature flags
- duplicate routes
- duplicate models
- migration issues

- frontend production builds
- backend test suite
- integration tests
- E2E tests
- Docker build
- Docker Compose startup
- health/readiness
- Celery worker/Beat startup

Perform a final architecture trace for these critical paths:

- 1 Patient booking through normal UI.
- 2 Patient booking through AI assistant.
- 3 EHR unknown outcome recovery.
- 4 Questionnaire completion.
- 5 Doctor appointment workflow.
- 6 Hospital Admin configuration.
- 7 Platform Admin hospital approval.
- 8 Operations reconciliation.
- 9 Notification delivery.
- 10 Document retrieval through authorized AI context.

For each trace verify that every layer is using the intended boundary and that no shortcut has been introduced.

Do not perform a large rewrite simply to make the repository look cleaner. Preserve working behavior. Fix only concrete architectural defects, security issues, correctness issues, duplicated critical logic, or maintainability problems that materially affect the product.

At the end, generate a final engineering report containing:

- architecture verified
- modules verified
- major defects fixed
- tests executed
- deployment checks executed
- known limitations
- intentionally deferred items
- recommended future work, clearly separated from current implementation

The final codebase should be a coherent healthcare access and scheduling platform with deterministic scheduling, verified EHR synchronization, tenant isolation, controlled AI capabilities, durable workflows, auditable operations, and maintainable application boundaries.